



浙江工业大学

数据结构课程设计

地图网络的图分析

学生姓名

田笑喧

学生学号

302023572097

所在班级

软件工程 2304

任课教师

王海霞

所在学院

计算机科学与技术学院

提交日期： 2024 年 12 月

地图网络的图分析

摘 要

本论文介绍了一个地图网络的图分析系统，该系统通过图形化界面模拟和分析地图网络，使用图论的概念来表示城市或地区之间的交通路线。系统实现了用户登录功能、图数据管理、图算法实现以及图形界面展示等核心模块。通过邻接矩阵和邻接表实现图结构，支持深度优先搜索(DFS)和广度优先搜索(BFS)等图遍历操作，并能够通过 Dijkstra 算法查找两点之间的最短路径。此外，系统还能够让用户添加或删除节点和边，查询特定节点的所有邻近节点，以及查询所有路径。系统的设计满足了教学和研究的需求，同时也提供了扩展功能的可能性，如地图网络的可视化展示，增强了用户的交互体验和系统的实用性。

关键字：地图网络分析、图论应用、SFML、图形用户界面、Dijkstra 算法、深度优先搜索（DFS）、广度优先搜索（BFS）、图结构、邻接矩阵、交互式系统、数据可视化

目 录

摘 要.....	I
目 录.....	1
第 1 章 课程设计与要求.....	2
1.1 问题描述.....	2
1.2 基本要求.....	2
1.3 实现提示.....	2
第 2 章 设计思路与分析.....	3
2.1 系统总体设计.....	3
2.1.1 系统框架选择.....	3
2.2 系统功能设计.....	3
2.2.1 用户登录模块.....	3
2.2.2 图数据管理及算法模块.....	4
2.2.3 图形界面展示及交互操作模块.....	4
2.3 本章小结.....	5
第 3 章 调试与分析.....	6
3.1 主要难点分析.....	6
3.1.1 图算法.....	6
3.1.2 文件读写与数据同步易错点.....	6
3.1.3 典型错误分析.....	6
3.2 本章小结.....	6
第 4 章 测试结果与分析.....	8
4.1 可视化建设.....	8
4.1.1 用户登录可视化.....	8
4.1.2 添加节点和删除节点可视化.....	10
4.1.3 添加边和删除边可视化.....	13
4.1.4 查询最短路径可视化.....	15
4.1.5 查询所有路径可视化.....	16
4.1.6 查询所有邻近节点.....	18
4.4 本章小结.....	19
第 5 章 项目总结.....	20
5.1 数据获取与分析.....	20
5.2 展望.....	20
参考文献.....	22
附录.....	23

第1章 课程设计与要求

1.1 问题描述

地图网络可以被看作是一个图，其中节点代表城市或地区，边代表城市或地区之间的交通路线。请设计一个程序来模拟地图网络，并使用图来表示这个网络。系统需要有用户登录功能，用户能够查看整个地图网络信息，添加/删除节点和边，查找特定节点的所有邻近节点，以及查找两个节点之间的最短路径，用于点到点的导航。

1.2 基本要求

1. 自己编程实现图结构，包括邻接矩阵与邻接表，及其相关功能，不允许使用标准模板类的图结构和函数。同时要求根据图的变化情况，进行相应的图遍历操作，包括但不限于深度优先搜索和广度优先搜索。测试时，各种情况都需要测试，并附上测试截图；
2. 要求采用类的设计思路，不允许出现类以外的函数定义，但允许友元函数。
3. 要求采用多文件方式：`.h` 文件存储类的声明，`.cpp` 文件存储类的实现，主函数 `main` 存储在另外一个单独的 `cpp` 文件中。如果采用类模板，则类的声明和实现都放在 `.h` 文件中。
4. 不强制要求采用类模板；不强制要求采用可视化窗口，但若采用可视化窗口可适当提高考核分数；要求源程序中有相应注释；
5. 要求测试例子要比较详尽，各种极限情况也要考虑到，测试的输出信息要详细易懂，表明各个功能的执行正确；

1.3 实现提示

1. 城市或地区信息(即城市或地区名称和交通路线)可以存储在文件中，当程序启动时，从文件中读取所有的城市或地区信息，并建立合适的图结构；
2. 验证过程时，需要根据城市或地区名称，遍历整个图，找到匹配的城市或地区，进行验证；更新交通路线时，也需要遍历图，找到匹配项后进行更新，同时更新文件中存储的交通路线。
3. 路径的搜索实现可以利用栈或队列结构
4. 添加城市或地区时，不仅需要在文件中添加，也需要在图中添加相应的节点；删除城市或地区时，也是如此。
5. 地图网络系统可以联系实际，扩展相应功能。例如，实现地图网络的可视化。

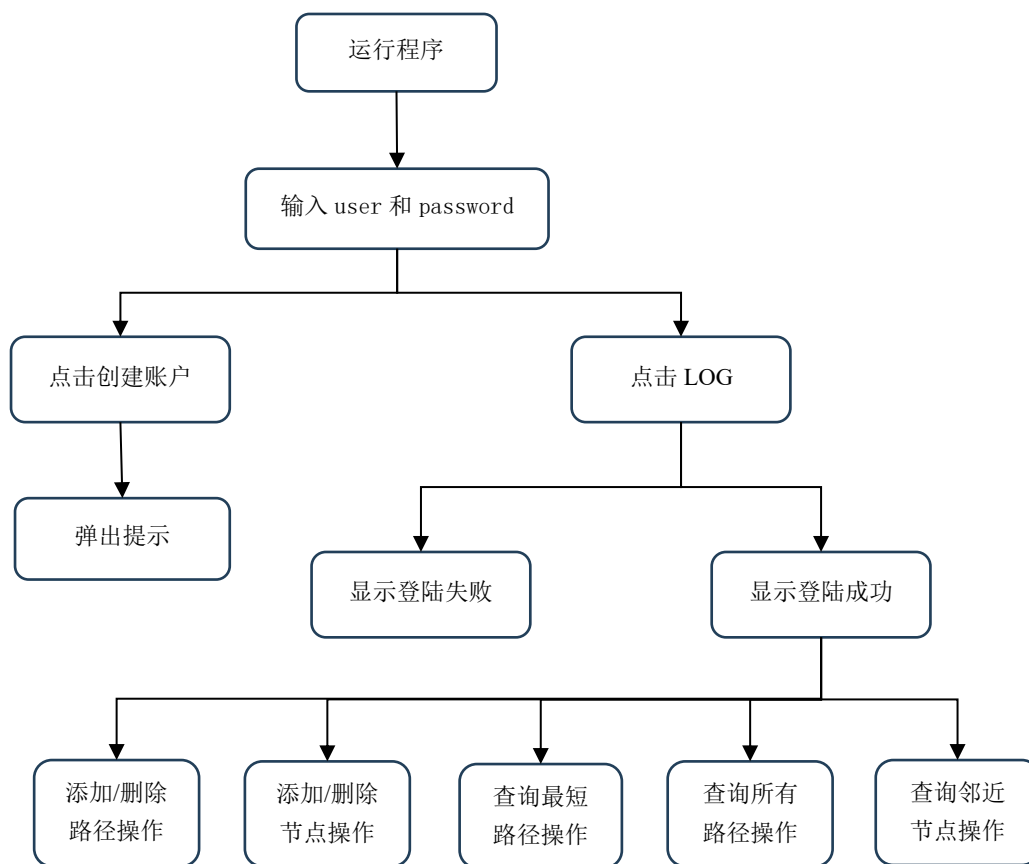
第 2 章 设计思路与分析

本章针对相关图形化界面的图管理工具进行了详细介绍,涉及到图形界面展示、用户交互、文件操作和图算法实现。

2.1 系统总体设计

2.1.1 系统框架选择

客户端-服务器架构: 本系统是一个单机应用, 主要运行在客户端, 没有明显的服务器端。所有的数据处理和图形渲染都在本地完成。



2.2 系统功能设计

2.2.1 用户登录模块

代码说明:

UserLogin 类在 UserLogin.cpp 和 UserLogin.h 中定义, 负责处理用户相关的操作。

createUser 方法用于创建新用户, 将用户名和密码存储在一个 map 中, 并写入到一个文件中。

verifyUser 方法用于验证用户名和密码是否匹配, 确保登录的合法性。

算法说明：

用户名和密码的存储使用了简单的文本文件，密码应该经过哈希处理以提高安全性。用户验证过程中，通过查找 `map` 中的用户名，并比对密码来确认用户身份。

作用：

用户登录模块是系统的入口点，确保只有经过身份验证的用户才能访问图管理工具的功能。这个模块是保护系统资源和数据不被未经授权访问的第一道防线。

与系统的关联：

验证通过后，用户可以进入系统进行图相关的操作，如查看、编辑图结构，运行图算法等。用户登录模块与图数据管理模块相互作用，因为不同的用户可能有权访问不同的图数据。

2.2.2 图数据管理及算法模块

代码说明：

`Graph` 类在 `Graph.cpp` 和 `Graph.h` 中定义，包含了图的基本操作和图算法的实现。

`addEdge` 和 `removeEdge` 方法用于修改邻接矩阵，从而添加或删除图中的边。

`shortestPath` 方法实现了 `Dijkstra` 算法，用于查找两点之间的最短路径。

`searchAllPaths` 和 `DFS` 方法实现了深度优先搜索，用于查找从一个节点到另一个节点的所有路径。

`adjacentVertices` 方法用于实现查找一个节点的所有邻近节点。

算法说明：

`Dijkstra` 算法通过维护一个优先队列来选择未访问的最近节点，更新距离，并记录路径。而这里距离默认为 1。

`DFS` 通过递归遍历所有可能的路径，记录下所有从起点到终点的路径。

作用：

图数据管理模块是系统的核心，负责图的创建、存储、修改和检索。图算法模块则在这个基础上提供算法支持，如路径查找、图遍历等，是实现图论功能的基础。

与系统的关联：

图数据管理模块与图形界面展示模块紧密协作，将图的内部数据结构（如邻接矩阵）转换为用户可以理解的视觉表示。

图算法模块的结果（如最短路径）可以通过图形界面展示模块直观地展示给用户，增强了算法的实用性和用户的交互体验。

2.2.3 图形界面展示及交互操作模块

代码说明：

`main.cpp` 中包含了使用 `SFML` 创建图形界面的代码，负责展示图和处理用户交互。

`drawMatrix` 函数用于将图的邻接矩阵状态绘制到窗口上，使用不同颜色表示边的存在与否。

程序中定义了多个函数实现按钮操作，如登录按钮、添加边按钮、删除边按钮等，每个按钮都对应一个特定的操作。

算法说明：

图形界面的渲染算法涉及到如何将图的数学表示（邻接矩阵）转换为视觉上

的表示，例如，使用不同颜色的点来表示节点和边。

事件处理算法涉及到如何响应用户的输入，例如鼠标点击或键盘输入，并根据这些输入调用相应的图操作。

作用：

图形界面展示模块提供了一个直观的用户界面，使用户能够方便地进行图的创建、编辑和分析。交互操作模块则响应用户的输入，如点击按钮或输入数据，触发相应的图操作。

与系统的关联：

图形界面展示模块是用户与系统交互的桥梁，它将复杂的图数据和算法结果以图形化的方式呈现给用户，使得非专业用户也能理解和使用图论工具。

交互操作模块通过监听用户的输入事件，如点击和键盘输入，来调用图数据管理及算法模块的功能，实现了用户意图与系统操作之间的转换。

2.3 本章小结

本章对系统的框架做了一个概述，简要介绍了用户登录模块、数据库管理及算法模块、图形界面展示及交互模块。这三个模块共同构成了一个完整的系统，每个模块都承担着不同的职责，但又相互依赖、相互协作。用户登录模块确保了系统的安全性，图数据管理及算法模块提供了图论的核心功能，而图形界面展示及交互操作模块则提升了用户体验，使得系统易于使用。这种模块化的设计不仅提高了系统的可维护性和可扩展性，也使得各个部分可以独立开发和测试，最终集成成一个协调一致的系统。通过这种方式，系统能够以高效、安全和用户友好的方式提供复杂的图论功能。

第 3 章 调试与分析

3.1 主要难点分析

3.1.1 图算法

3.1.1.1 图算法的正确实现

Dijkstra 算法和 DFS 算法需要正确处理图的遍历和路径记录，任何逻辑错误都可能导致错误的结果。

邻接矩阵的更新需要同步，以确保图的一致性。

3.1.1.2 算法具体实施

Dijkstra 算法：

使用优先队列选择最近的未访问节点，并更新其邻接节点的距离。

记录每个节点的前驱节点以重建最短路径。

DFS 算法：

使用递归或栈实现深度优先搜索，记录访问路径并保存所有可能的路径。

3.1.2 文件读写与数据同步易错点

图数据的持久化需要在每次修改后更新文件，这涉及到文件读写操作的准确性和数据同步问题。

使用 C++ 文件流进行图数据的读写，确保格式正确并处理可能的 I/O 错误

3.1.3 典型错误分析

3.1.3.1 邻接矩阵索引越界

在图操作中，如 `addEdge` 和 `removeEdge`，如果传入的节点索引超出了邻接矩阵的大小，会导致越界访问。

在 DFS 回溯中，如果找寻已删除节点，会直接对程序进行报错。

3.1.3.2 文件路径错误

在 `Graph::updateFile` 和 `UserLogin::createUser` 中，硬编码的文件路径可能导致在不同环境下无法找到文件。

在 `tasks.json` 中未添加 `UserLogin.cpp` 和 `Graph.cpp` 路径，会导致程序运行时提示无法编译但没有报错信息。

3.1.3.3 内存管理

在处理大量图数据时，如果没有正确管理内存，可能会导致内存泄漏。

3.2 本章小结

针对本章提出的程序调试与分析，我们探讨了图形界面与事件处理、图算法的正确实现以及文件读写与数据同步这三个主要难点。在算法具体实施方面，我们详细分析了 Dijkstra 算法、DFS 算法和文件读写操作的关键步骤。错误分析部分，我们识别了邻接矩阵索引越界、文件路径错误和内存管理这几个易错点。典

型错误分析则涵盖了逻辑错误、并发访问问题、图形界面响应不及时和异常处理不足等问题。通过这些分析，我们可以更全面地理解程序的潜在问题，并采取相应的措施来提高程序的稳定性和用户体验。

第 4 章 测试结果与分析

4.1 可视化建设

4.1.1 用户登录可视化

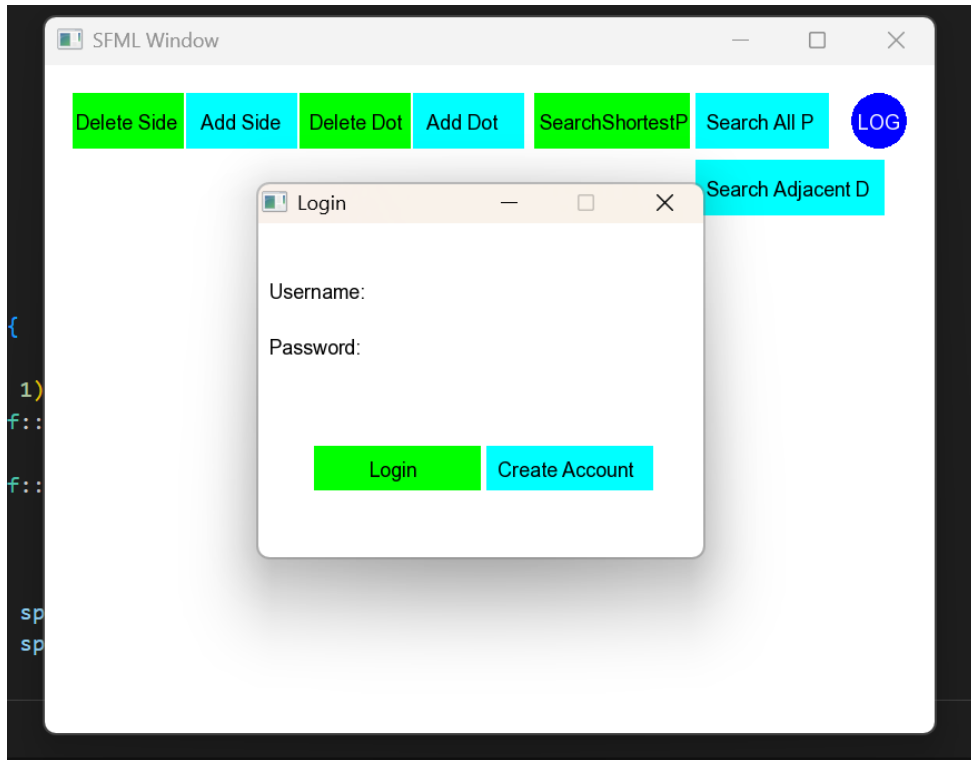


图 4. 1. 1-1

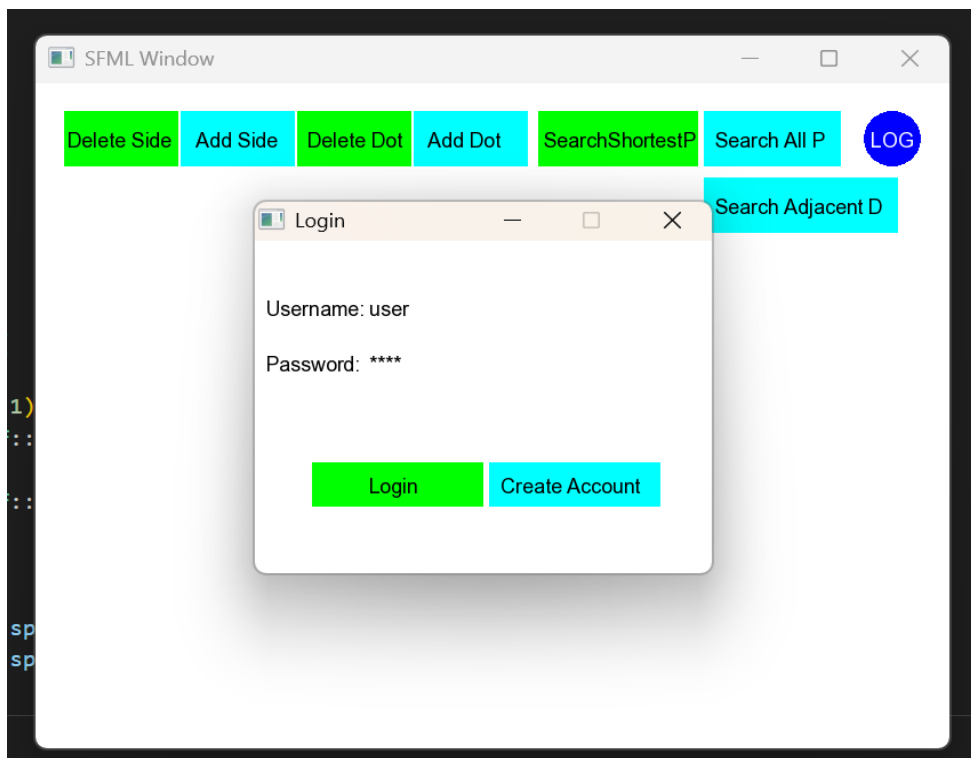


图 4.1.1-2

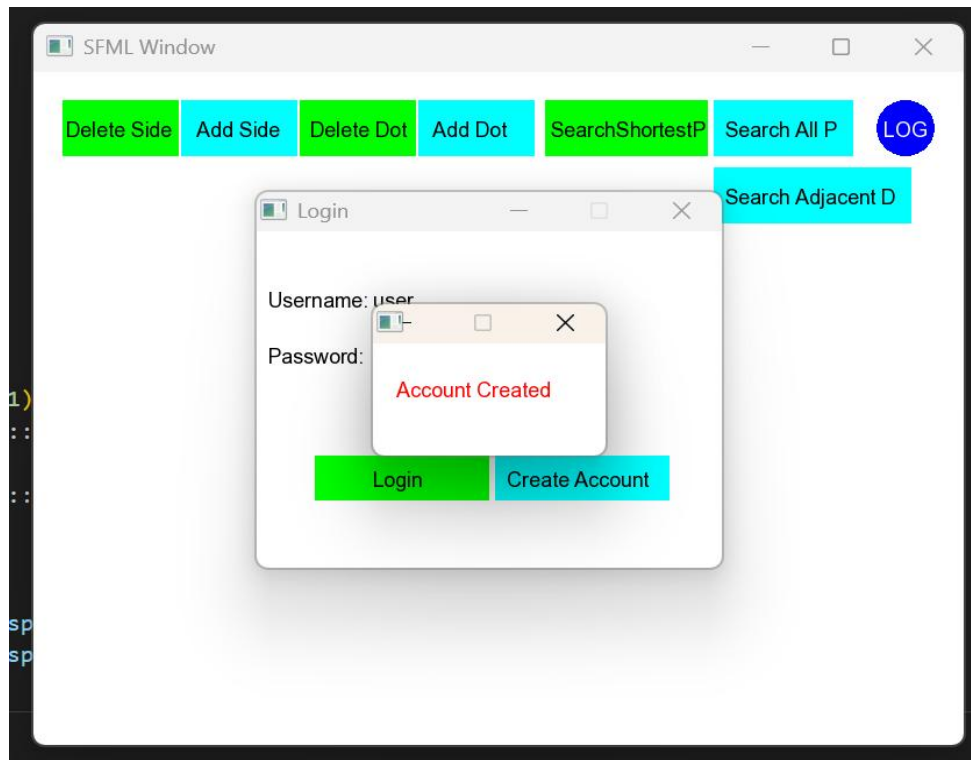
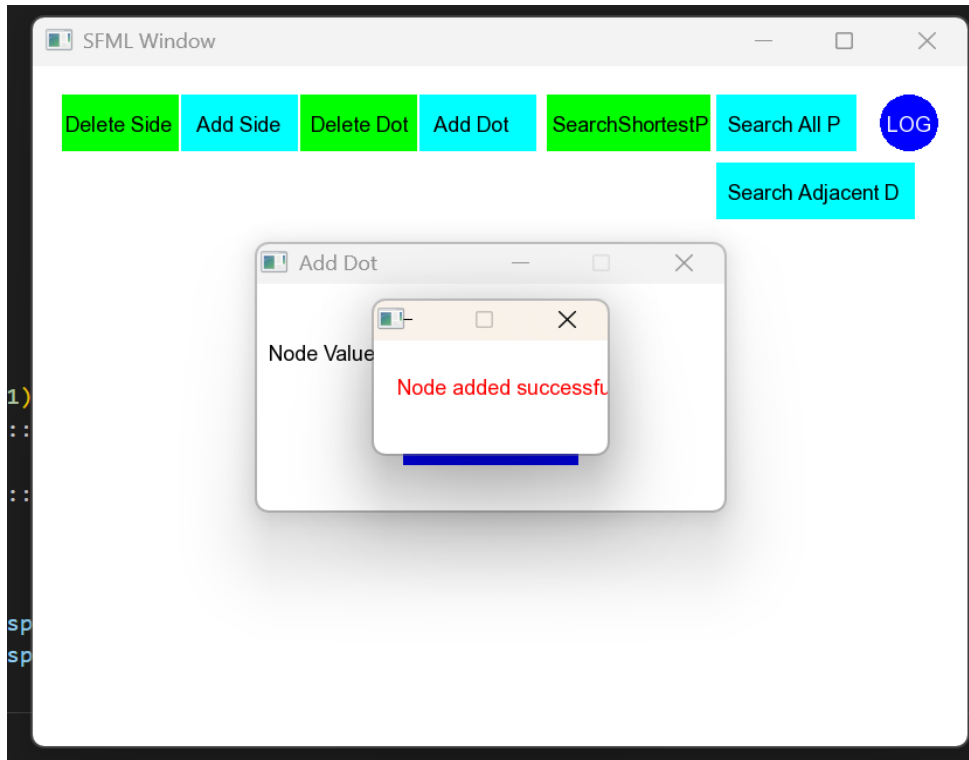


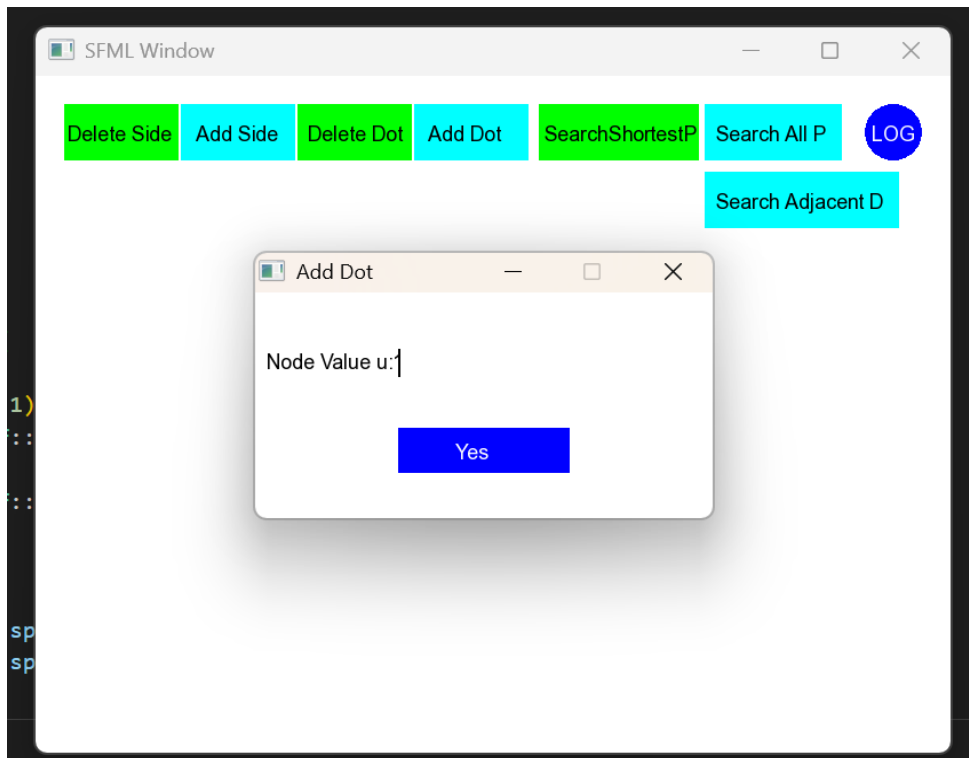
图 4.1.1-3

图 1 为用户登录界面，图 2 为用户输入显示，图 3 为用户账号创建界面。点击 Login 后，登录成功会直接关闭窗口，登陆失败会弹出错误提示。

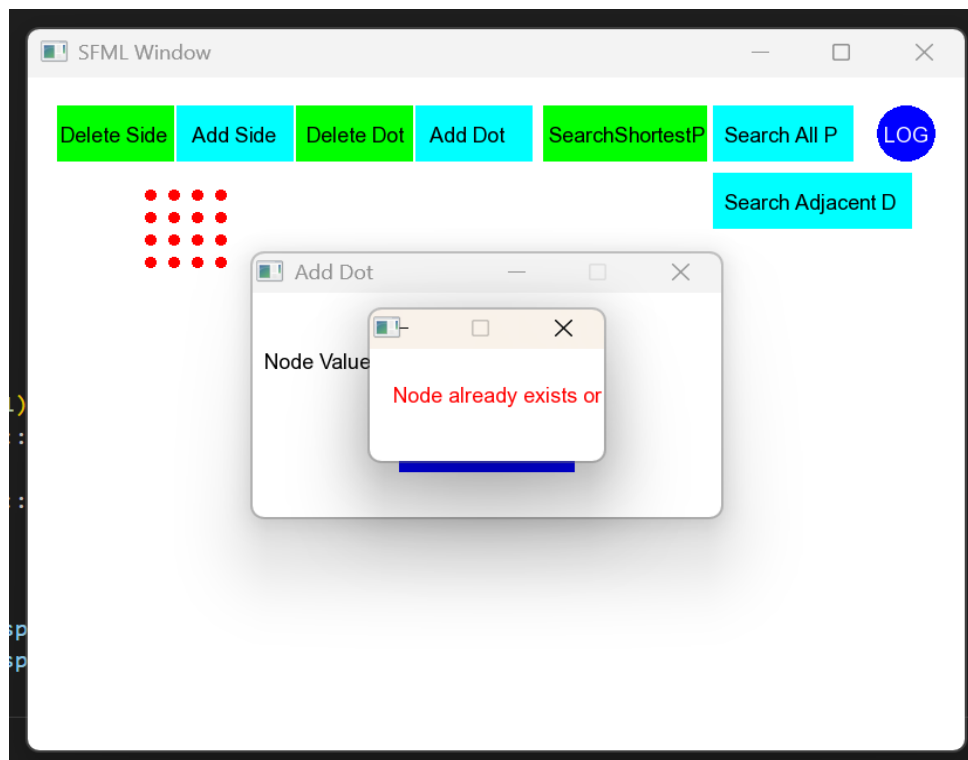
4.1.2 添加节点和删除节点可视化



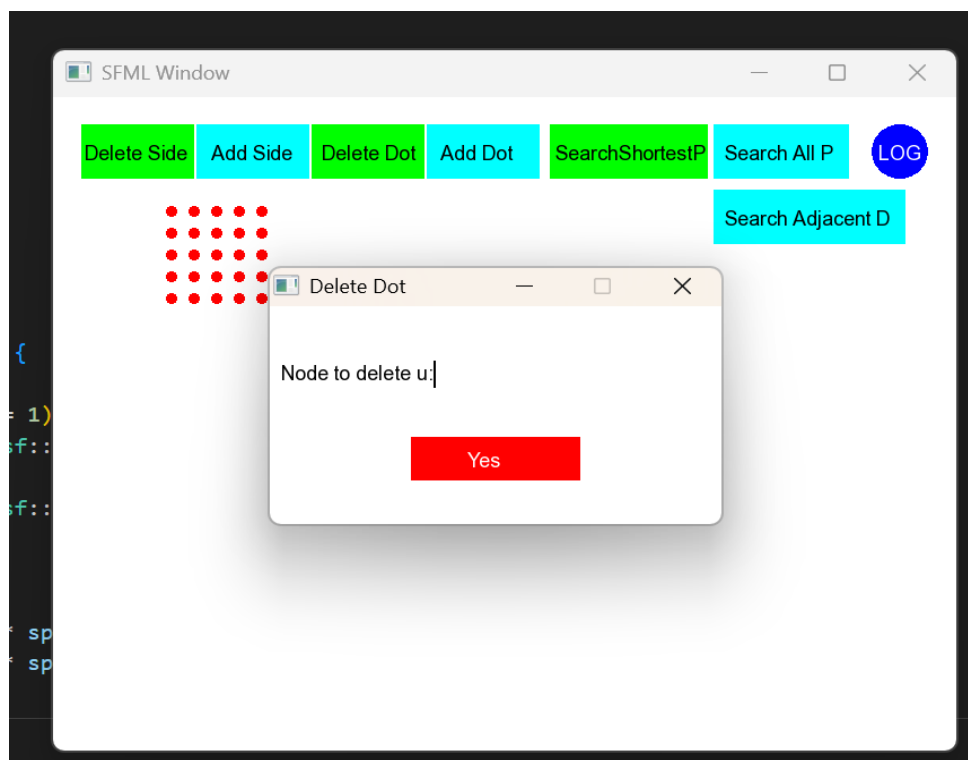
图片 4.1.2-1



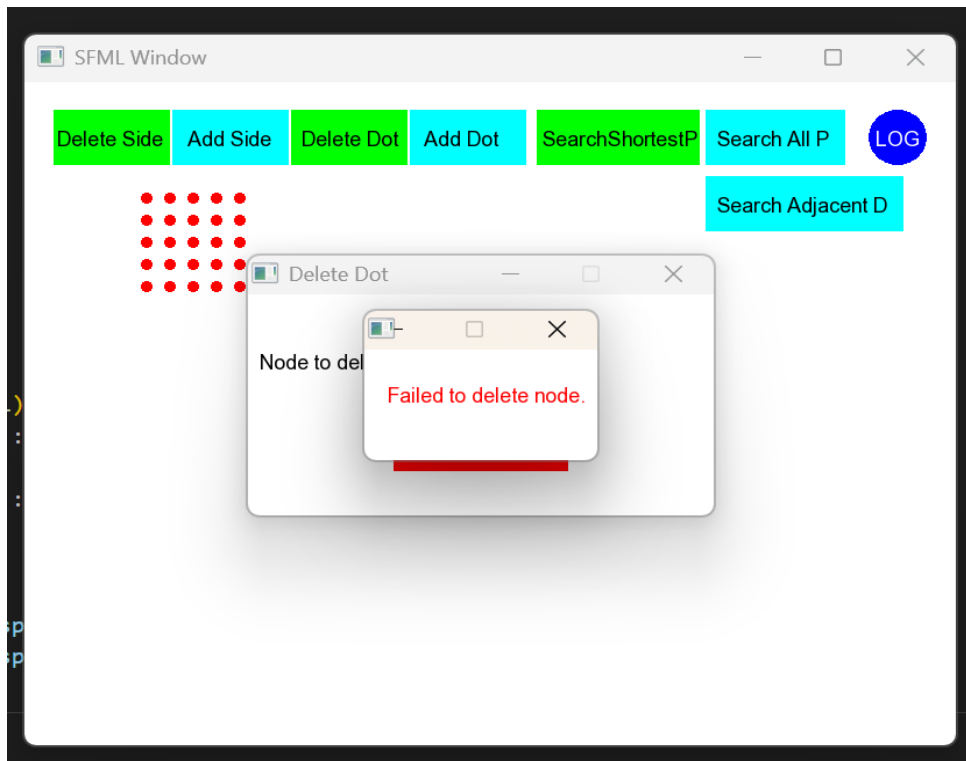
图片 4.1.2-2



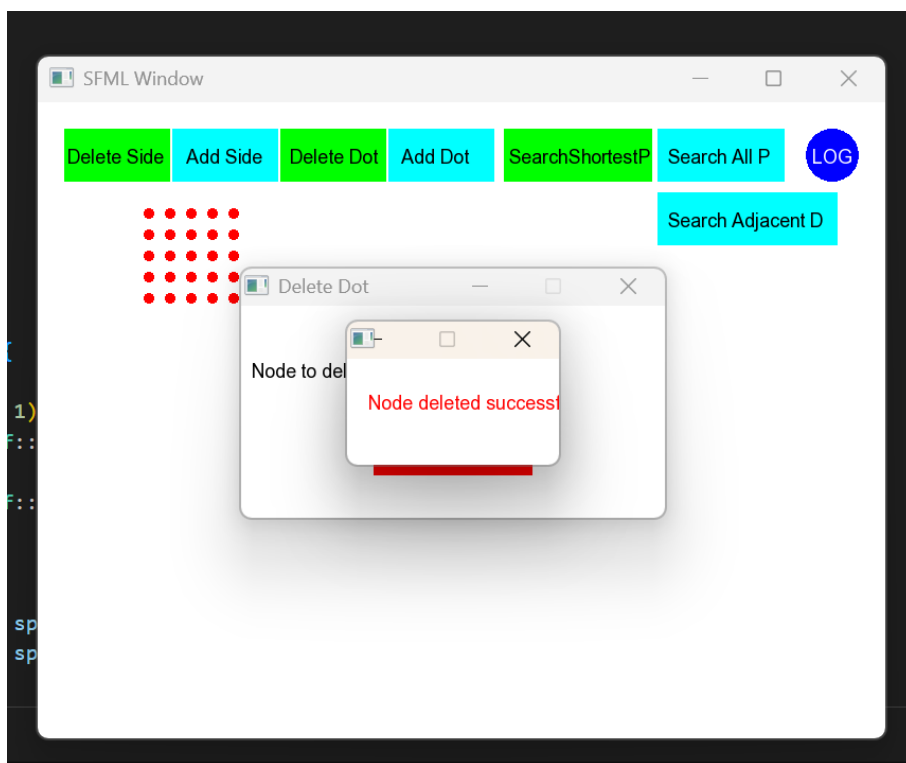
图片 4.1.2-3



图片 4.1.2-4



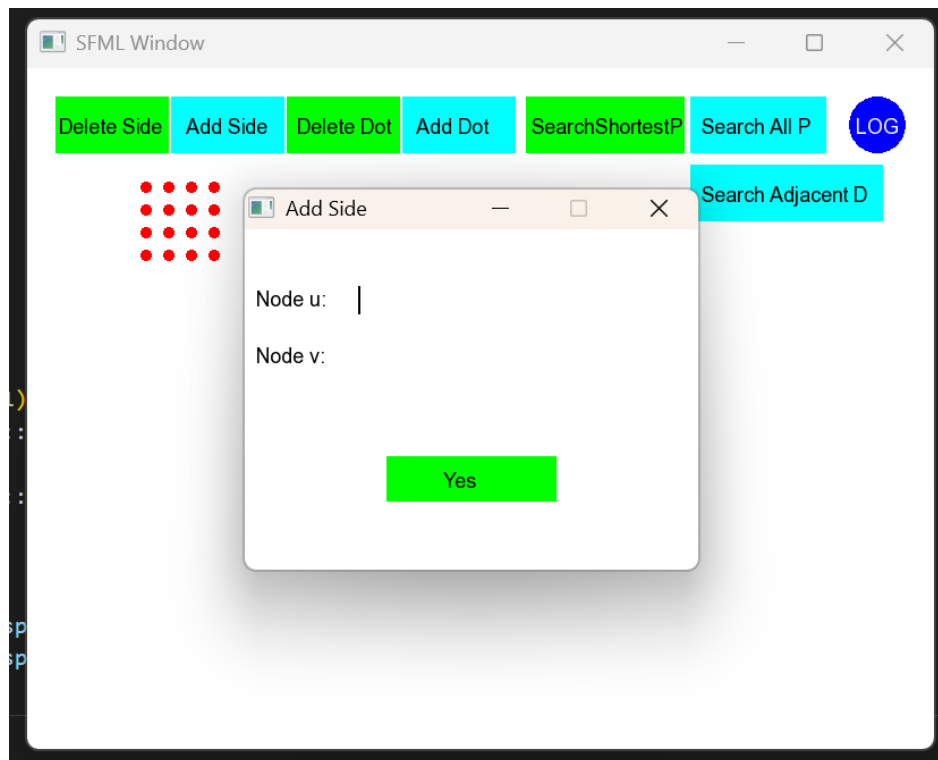
图片 4.1.2-5



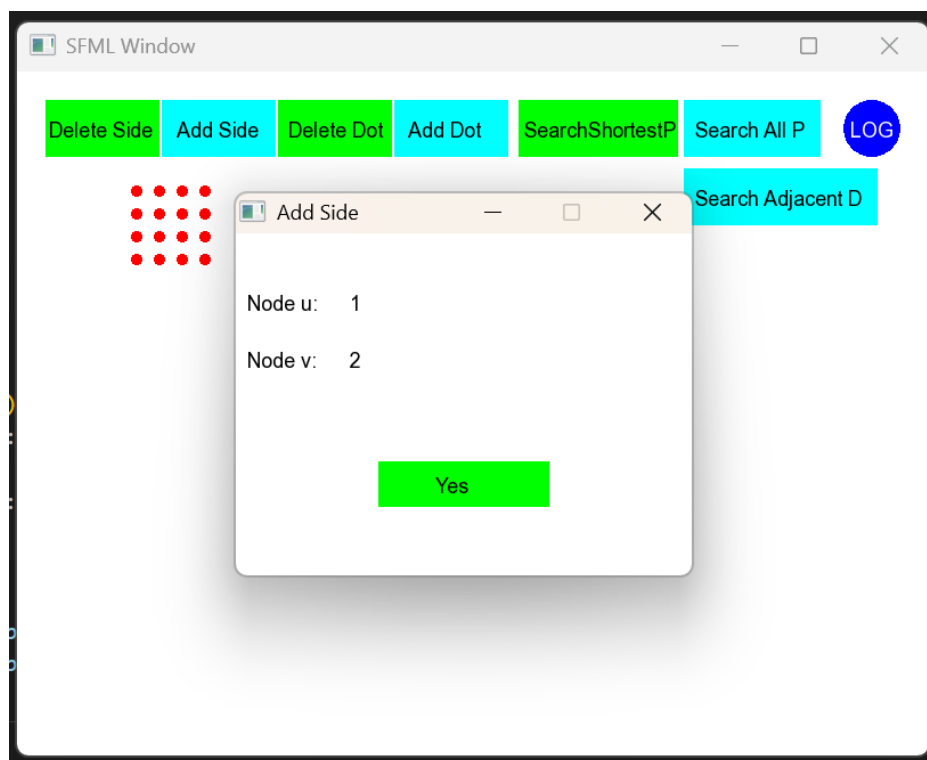
图片 4.1.2-6

添加节点成功后会弹出提示框，关闭提示框后会自动刷新界面，显示节点及边的状态。

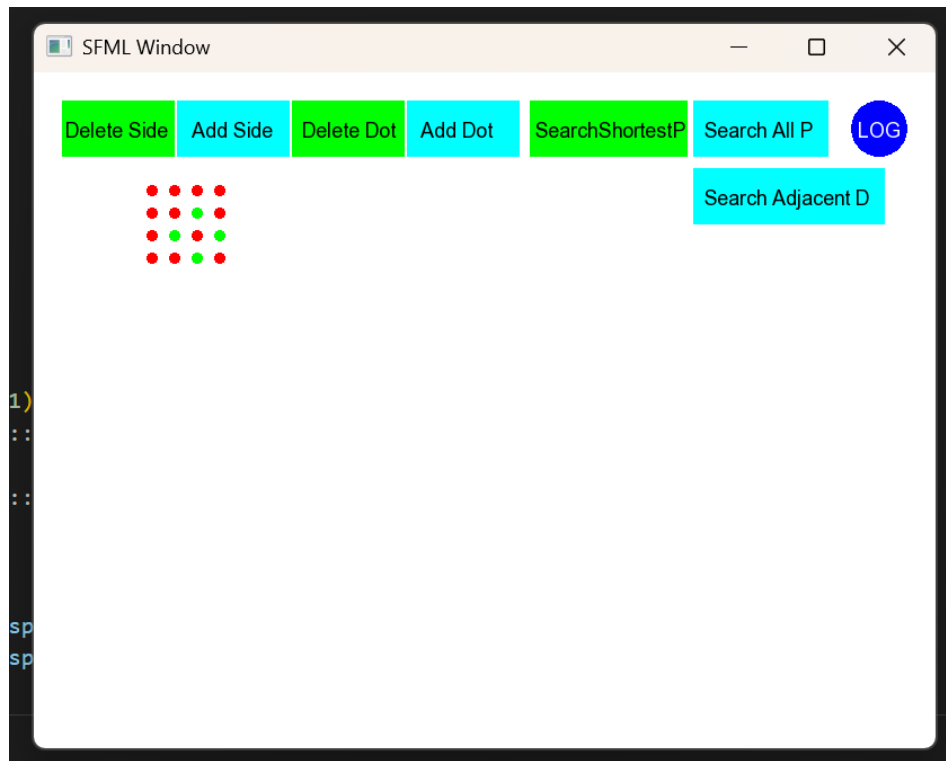
4.1.3 添加边和删除边可视化



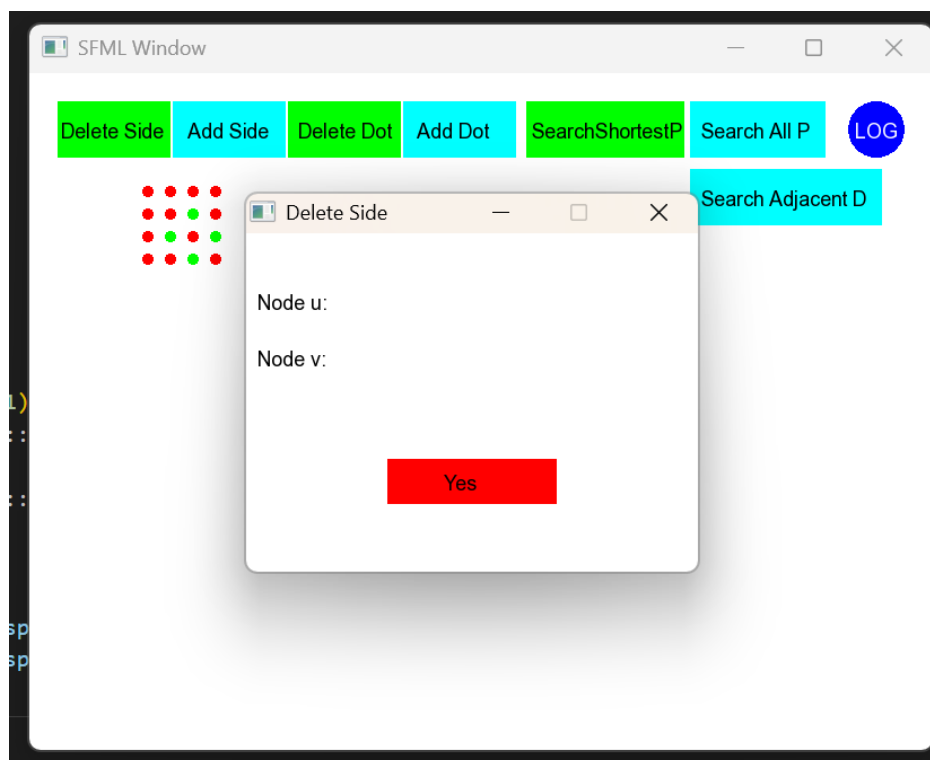
图片 4.1.3-1



图片 4.1.3-2



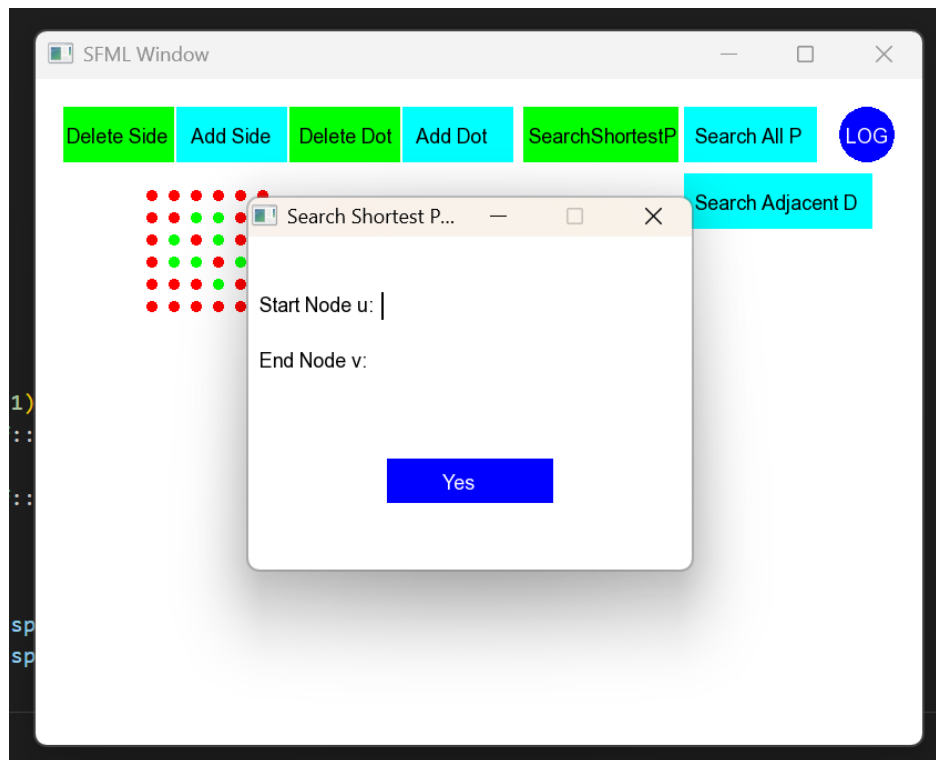
图片 4.1.3-3



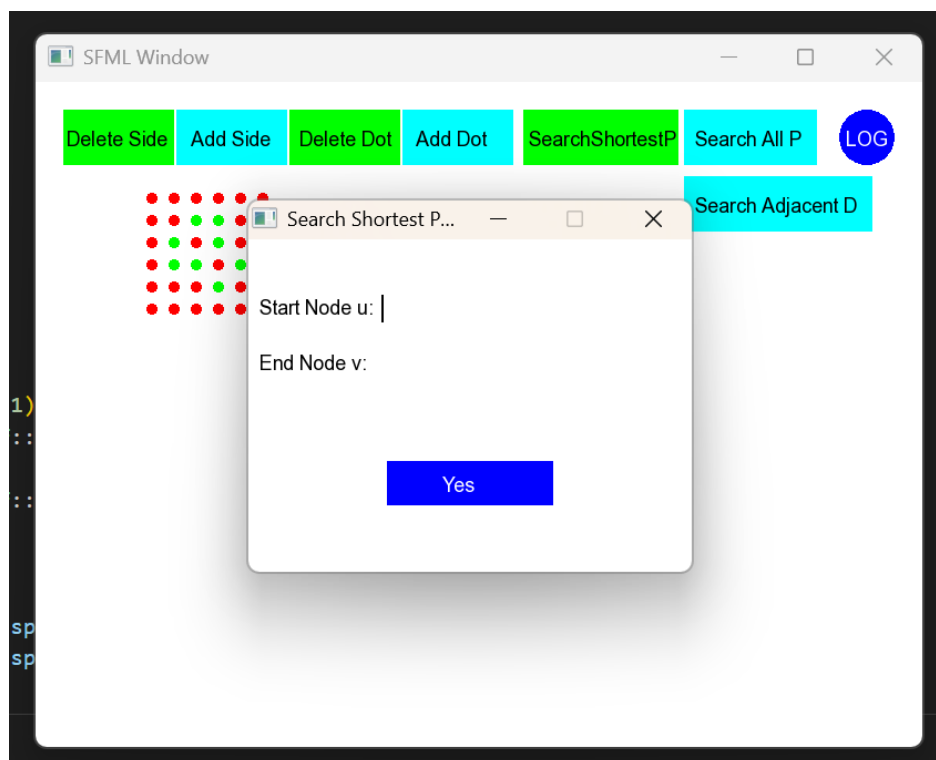
图片 4.1.3-4

添加边成功后会弹出提示框，关闭提示框后会自动刷新界面，显示节点及边的状态。

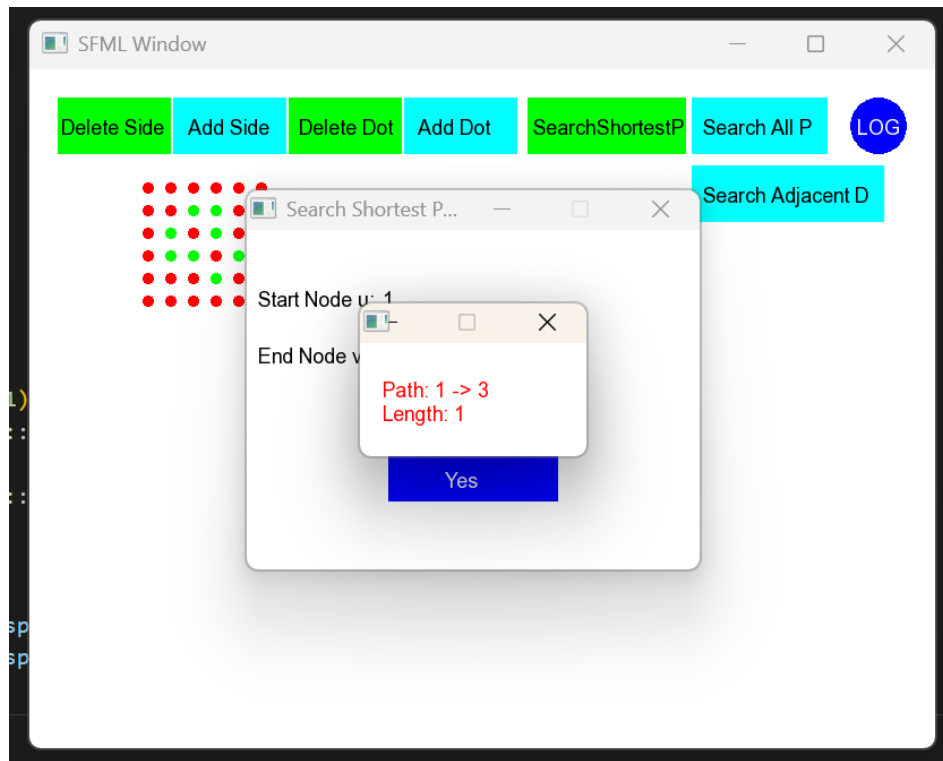
4.1.4 查询最短路径可视化



图片 4.1.4-1



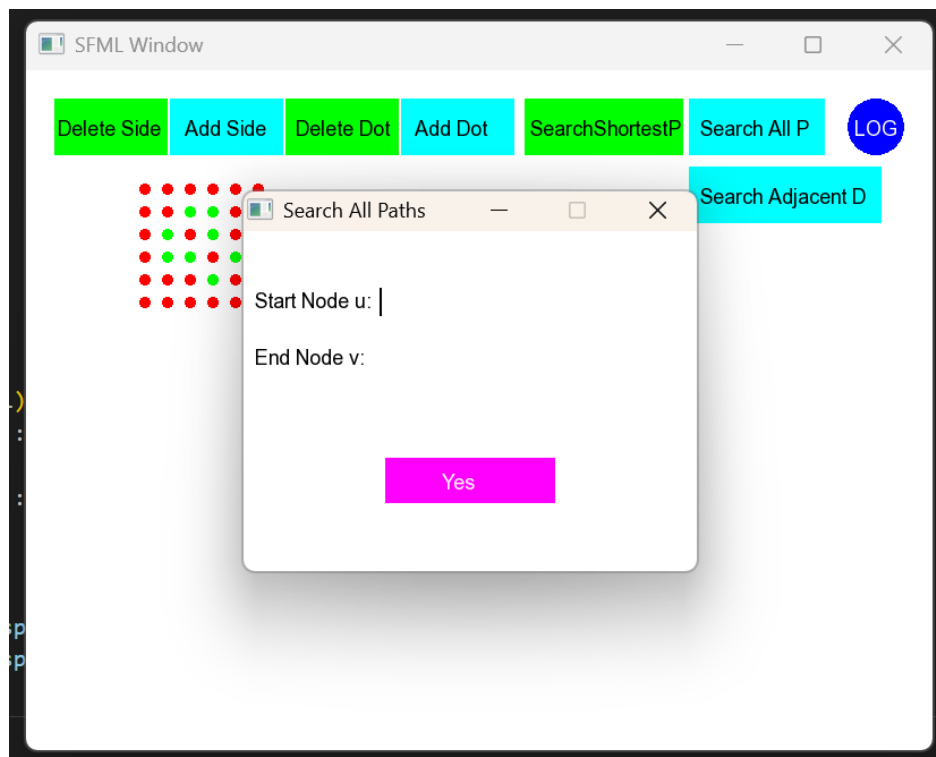
图片 4.1.4-2



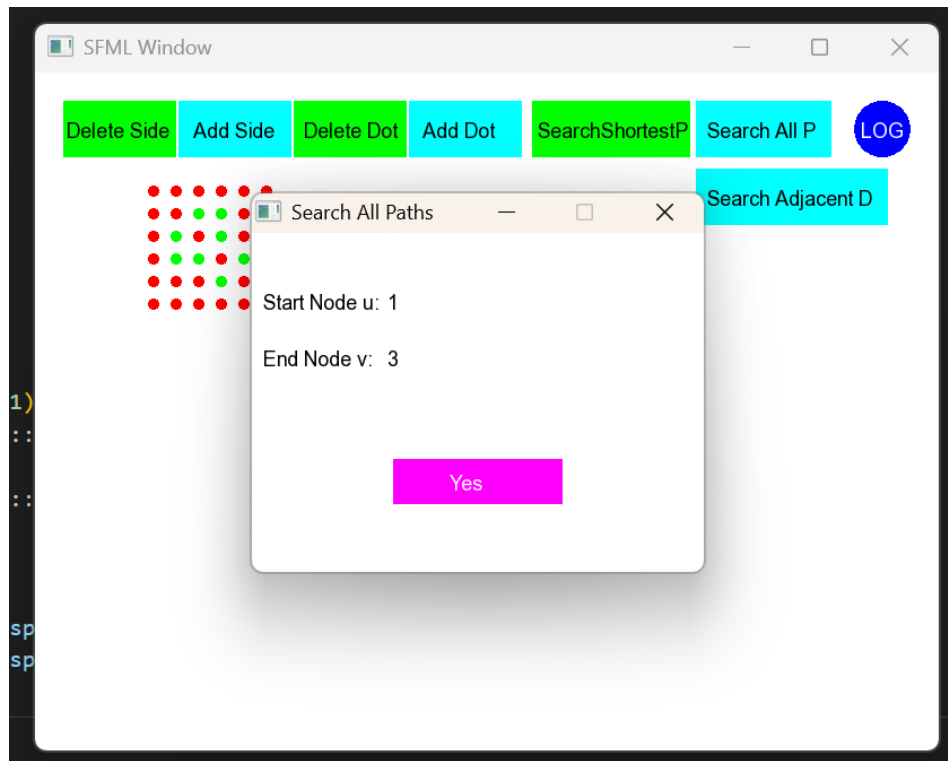
图片 4.1.4-3

输入起始节点以及终止节点后，会自动输出路径及最短路径长度。

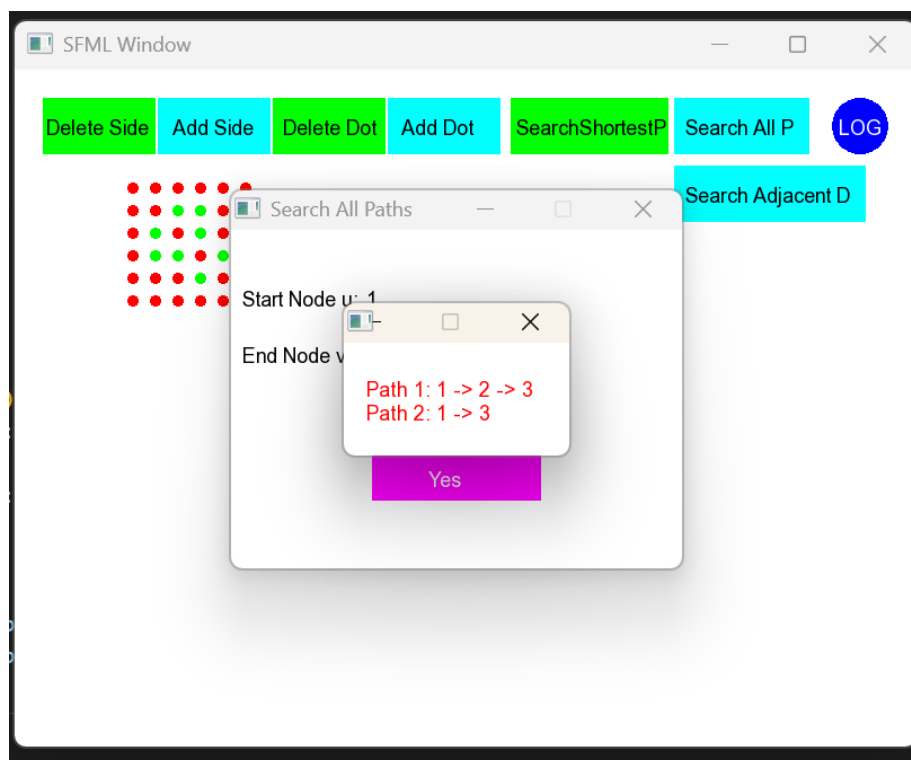
4.1.5 查询所有路径可视化



图片 4.1.5-1



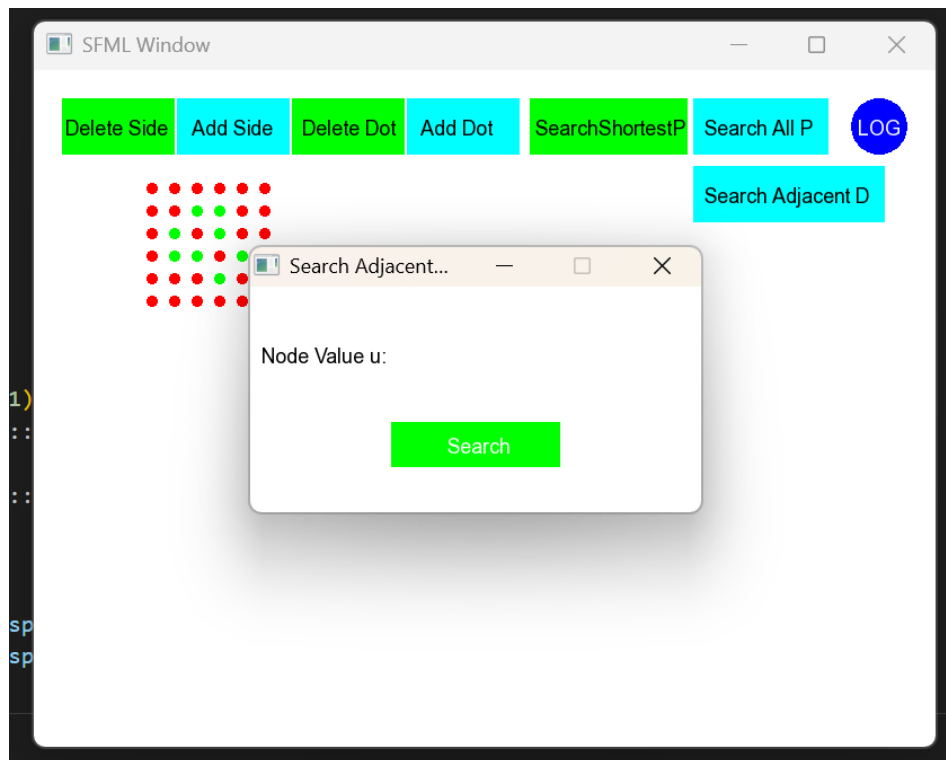
图片 4.1.5-2



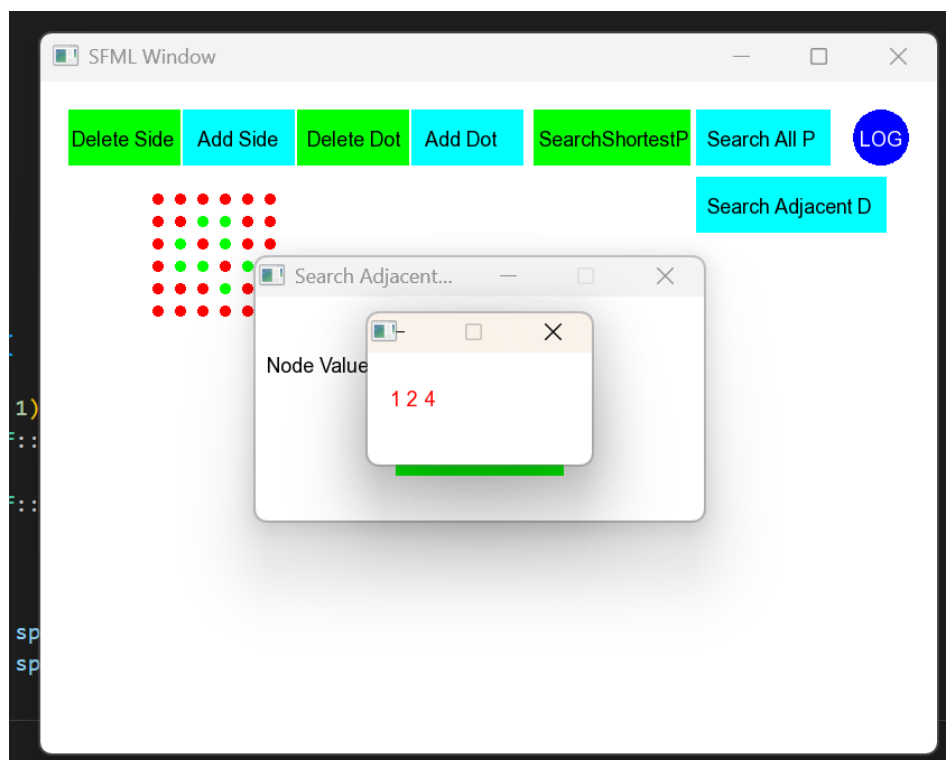
图片 4.4.5-3

输入起始节点以及终止节点后，会自动输出所有路径。

4.1.6 查询所有邻近节点



图片 4.1.6-1



图片 4.1.6-2

输入节点后，会自动输出所有邻接节点。

4.4 本章小结

在本章中，主要展示了输出、输入和各种操作的可视化。用两个节点交叉处的红点表示节点无路径，绿点表示节点有路径。

第 5 章 项目总结

我设计的地图网络图分析系统是一个综合性的解决方案，旨在通过图形化界面模拟和分析地图网络。该系统完全符合任务书的要求，提供了用户登录功能、图数据管理、图算法实现以及图形界面展示等核心模块。系统采用邻接矩阵和邻接表来实现图结构，支持深度优先搜索(DFS)和广度优先搜索(BFS)等图遍历操作，并且能够通过 Dijkstra 算法查找两点之间的最短路径。此外，系统还能够让用户添加或删除节点和边，查询特定节点的所有邻近节点，以及查询所有路径。这些功能不仅满足了基本的教学需求，还为用户提供了扩展功能的可能性，如地图网络的可视化展示，增强了用户的交互体验和系统的实用性。

5.1 数据获取与分析

在数据获取与分析方面，系统通过读取存储城市或地区信息的文件来构建图结构，这些信息包括城市名称和它们之间的交通路线。系统能够处理各种图的变化情况，并进行相应的图遍历操作。在测试过程中，系统能够详尽地展示各种极限情况下的功能执行，确保了数据的准确性和算法的有效性。系统还提供了数据持久化功能，能够将图的修改实时更新到文件中，确保了数据的一致性和完整性。此外，系统还能够将图的内部数据结构转换为用户可以理解的视觉表示，增强了数据的可读性和分析的直观性。通过这些功能，用户可以轻松地获取和分析地图网络中的关键信息。

5.2 展望

展望未来，系统将在以下几个方面进行改进和提高：

1. 算法优化：
系统将引入更高效的图算法，如 A*搜索算法和 Floyd-Warshall 算法，以提供更全面的图论分析工具。这些算法将优化路径查找和图遍历的性能，特别是在处理大规模图数据时。
2. 系统性能：
通过优化图遍历和路径搜索算法，提高系统处理大规模图数据的能力，减少计算时间和资源消耗。这将使得系统在处理复杂的地图网络时更加稳定和高效。
3. 用户界面改进：
进一步改善图形用户界面，使其更加直观和易用。增加更多的用户指导和帮助文档，降低用户的学习曲线，使得即使是非专业的用户也能快速上手。
4. 数据可视化增强：
增强数据可视化功能，提供更多定制化的视图选项，使用户能够根据需要调整图的显示方式，更好地理解和分析图数据。这将包括更多的颜色编码、节点和边的自定义样式以及交互式图表。
5. 扩展功能：
系统将考虑集成更多的功能，如实时交通流量分析、多模式交通网络支

持等，以满足更广泛的应用场景。

6. 用户体验研究：

通过用户反馈和使用数据分析，不断优化用户体验，确保系统的设计和功能能够满足用户的实际需求。

通过这些改进，系统将能够更好地服务于图论教育和研究领域，为用户提供一个更加强大和灵活的工具，同时也为地图网络分析提供了新的视角和方法。

参考文献

无。

附录

Graph.h

```
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <limits>
#include <cstring>

using namespace std;

const int maxv = 100;    // 最大节点数

class Graph {
public:
    int edges[maxv][maxv]; // 邻接矩阵
    int n, e; // 图的节点数和边数
    string veks[maxv]; // 存储节点名称

    Graph(); // 构造函数
    void CreatGraph(int a[][maxv], int n, int e); // 创建图的邻接矩阵
    void addEdge(int u, int v); // 添加边
    void removeEdge(int u, int v); // 删除边
    bool addDot(int u); // 添加节点
    bool deleteNode(int index); // 删除节点
    string shortestPath(int start, int end); // 查找最短路径
    string searchAllPaths(int start, int end); // 查找所有路径
    string adjacentVertices(int vertex); // 查找邻接节点
    void updateFile(const string& filename); // 更新文件中的图数据
    void DFS(int v, int end, int pathIndex[], int index, char paths[maxv][100], int
    &count, bool visited[]); // 深度优先搜索
    void BFS(int v); // 广度优先搜索
    void loadGraphFromFile(const string& filename);
```

```
};

#endif

Graph.cpp
#include "Graph.h" //包含自定义头文件 Graph.h，其中声明了类 Graph 的定义
和方法。
#include <queue> // 包含 C++标准库头文件 queue，用于队列操作

Graph::Graph() {
    n = 0; // 初始化节点数为 0
    e = 0; // 初始化边数为 0
    // 初始化邻接矩阵
    for (int i = 0; i < maxv; i++) {
        for (int j = 0; j < maxv; j++) {
            edges[i][j] = 0; // 初始化邻接矩阵中的每个值为 0，表示无边
        }
    }
}

void Graph::CreatGraph(int a[][maxv], int n, int e) { //根据给定的邻接矩阵数组
a 创建图
    this->n = n; // 设置节点数
    this->e = e; // 设置边数
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            edges[i][j] = a[i][j]; // 复制邻接矩阵
        }
    }
}

void Graph::addEdge(int u, int v) { // 添加无向边
    edges[u][v] = 1; // 在邻接矩阵中添加边 (u, v)
    edges[v][u] = 1; // 添加对称边 (v, u) (无向图)
    e++; // 边数加 1

    // 更新文件
    updateFile("D:/vscode_wenjian/SFML/user.txt");
}
```

```
}

void Graph::removeEdge(int u, int v) { // 删除无向边
    edges[u][v] = 0; // 删除边 (u, v)
    edges[v][u] = 0; // 删除对称边 (v, u)
    e--; // 边数减 1

    // 更新文件
    updateFile("D:/vscode_wenjian/SFML/user.txt");
}

void Graph::updateFile(const string& filename) {
    ifstream readFile(filename); // 以读取模式打开文件，读取第一行
    if (!readFile.is_open()) {
        cerr << "Failed to open file for reading!" << endl; // 打印错误信息
        return;
    }
    string firstLine;
    getline(readFile, firstLine); // 读取第一行内容

    readFile.close(); // 关闭读取模式的文件

    ofstream writeFile(filename); // 以写入模式打开文件
    if (!writeFile.is_open()) {
        cerr << "Failed to open file for updating!" << endl; // 打印错误信息
        return;
    }

    writeFile << firstLine << endl; // 写入第一行（保持原样）

    writeFile << n << " " << e << endl; // 写入节点数和边数，从第二行开始

    // 写入邻接矩阵
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            writeFile << edges[i][j] << " ";
        }
        writeFile << endl;
    }
}
```

```

    }

    writeFile.close(); // 关闭文件
}

string Graph::shortestPath(int start, int end) { //使用 Dijkstra 算法计算两点之间的最短路径。
    int dist[maxv]; // 距离数组, 存储到每个节点的最短路径长度
    int prev[maxv]; // 前驱节点数组, 存储最短路径的前驱节点
    bool visited[maxv] = {false}; // 记录节点是否已访问
    fill(dist, dist + maxv, numeric_limits<int>::max()); // 距离初始化为无穷大
    dist[start] = 0; // 起点到自身距离为 0

    // Dijkstra 算法的实现
    for (int i = 0; i < n; i++) {
        int u = -1;
        int minDist = numeric_limits<int>::max();
        for (int j = 0; j < n; j++) {
            if (!visited[j] && dist[j] < minDist) {
                u = j; // 找到距离最近且未访问的节点
                minDist = dist[j];
            }
        }

        if (u == -1) break; // 没有可达节点
        visited[u] = true; // 标记为访问

        for (int v = 0; v < n; v++) {
            if (edges[u][v] != 0 && !visited[v] && dist[u] + edges[u][v] < dist[v])
            {
                dist[v] = dist[u] + edges[u][v]; // 更新距离
                prev[v] = u; // 记录前驱节点
            }
        }
    }

    // 回溯找到最短路径
    string path;

```

```

int current = end;
while (current != start) {
    path = to_string(current) + path; // 将当前节点添加到路径字符串
    if (current != start) {
        path = " -> " + path; // 添加箭头
    }
    current = prev[current]; // 回溯到前驱节点
}
path = to_string(start) + path;

// 如果路径不存在
if (path.empty() || prev[end] == -1) {
    return "No path found"; // 返回错误信息，无路径
}

// return 最短路径和长度
stringstream ss;
ss << "Path: " << path << "\nLength: " << dist[end];
return ss.str();
}

string Graph::adjacentVertices(int vertex) {
    stringstream ss;
    ss << ""; // 初始化空字符串
    cout << "Adjacent vertices to " << vertex << ": ";
    for (int i = 0; i < n; i++) { // 遍历节点
        if (edges[vertex][i] != 0) { // 如果存在边，说明是邻接节点
            ss << i << " "; // 将邻接节点加入结果
        }
    }
    return ss.str(); // return 邻接节点字符串
}

string Graph::searchAllPaths(int start, int end) {
    const int maxPathLength = 100; // 假设路径长度不超过 100
    char paths[maxv][maxPathLength]; // 存储所有路径
    int pathIndex[maxv]; // 存储当前路径的索引
    int count = 0; // 统计路径数

```

```

bool visited[maxv] = {false}; // 标记数组，记录节点是否已访问

// 调用 DFS 来生成所有路径
DFS(start, end, pathIndex, 0, paths, count, visited); // 深度优先搜索

// 拼接所有路径为字符串返回
stringstream sss;
sss << "";
for (int i = 0; i < count; ++i) {
    sss << "Path " << i + 1 << ": " << paths[i] << "\n"; // 将每条路径写入
}

if (count == 0) { // 如果没有路径
    sss << "No path found from " << start << " to " << end ;
}
return sss.str();
}

void Graph::DFS(int v, int end, int pathIndex[], int index, char paths[maxv][100], int
&count, bool visited[]) {
    visited[v] = true; // 标记当前节点已访问
    pathIndex[index] = v; // 将当前节点加入路径

    // 如果当前节点是终点
    if (v == end) { // 到达终点
        char path[100]; // 将路径转换为字符串
        path[0] = '\0'; // 初始化为空字符串
        for (int i = 0; i <= index; ++i) { // 遍历路径索引
            int node = pathIndex[i];
            if (i > 0) {
                strcat(path, "-> "); // 添加箭头符
            }
            char buffer[10];
            sprintf(buffer, "%d", node); // 将节点编号转为字符串
            strcat(path, buffer); // 拼接节点编号到路径
        }
        strcpy(paths[count++], path); // 将路径保存到`paths`数组
    }
}

```

```

//如果当前节点不是终点，递归访问所有邻接且未访问的节点
else {
    for (int i = 0; i < n; i++) {
        if (edges[v][i] != 0 && !visited[i]) { // 如果有边且未访问
            DFS(i, end, pathIndex, index + 1, paths, count, visited); // 递归调用
        }
    }
    visited[v] = false; // 回溯时取消访问标记
}

void Graph::BFS(int v) { //广度优先搜索，按层次遍历图
    bool visited[maxv] = {false}; // 记录访问过的节点
    queue<int> q;

    visited[v] = true; // 标记起始节点已访问
    q.push(v); // 将起始节点入队

    //遍历队列
    while (!q.empty()) {
        int u = q.front(); // 取队首元素
        q.pop(); // 出队
        cout << vexts[u] << " "; // 打印当前节点

        //遍历所有邻接节点
        for (int i = 0; i < n; i++) {
            if (edges[u][i] != 0 && !visited[i]) { // 如果有边且未访问
                visited[i] = true; // 标记为已访问
                q.push(i); // 入队
            }
        }
    }
}

bool Graph::addDot(int u) { //向图中添加一个新节点
    for (int i = 0; i < n; i++) {

```

```

        if (vexs[i] == to_string(u)) { // 检查是否已存在该节点
            return false; // 节点已存在，返回失败
        }
    }
    if (n < maxv) { // 如果节点数未达到上限
        vexs[n] = to_string(u); // 将新节点编号加入节点数组
        for (int i = 0; i < maxv; i++) {
            edges[n][i] = 0; // 初始化新节点的行
            edges[i][n] = 0; // 初始化新节点的列
        }
        n++; // 更新节点数
        return true;
    }

    // 更新文件
    updateFile("D:/vscode_wenjian/SFML/user.txt");

    return false; // 节点数已达上限，返回失败
}

bool Graph::deleteNode(int index) { // 从图中删除指定节点
    if (index < 0 || index >= n) { // 检查节点索引是否有效
        cout << "Node index is out of range." << endl;
        return false;
    }

    // 删除与该节点相关的所有边
    for (int i = 0; i < n; i++) {
        edges[index][i] = 0; // 删除行
        edges[i][index] = 0; // 删除列
    }
    n--; // 节点数减少

    // 删除节点后，重新排列节点
    for (int i = index; i < n; i++) { // 从删除的索引开始，向前覆盖
        vexs[i] = vexs[i + 1]; // 节点数组左移
        for (int j = 0; j < maxv; j++) {

```



```

        edges[i][j] = edges[i + 1][j]; // 邻接矩阵左移
        edges[j][i] = edges[j][i + 1]; // 邻接矩阵上移
    }
}
return true;
updateFile("D:/vscode_wenjian/SFML/user.txt"); // 更新文件
}

void Graph::loadGraphFromFile(const string& filename) { // 从文件加载图数据
    ifstream file(filename); // 打开文件
    if (!file.is_open()) {
        cerr << "Failed to open file for loading graph!" << endl;
        return;
    }

    // 读取节点数和边数
    file >> n >> e; // 从文件中读取节点数和边数

    // 读取邻接矩阵
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            file >> edges[i][j];
        }
    }
    file.close(); // 关闭文件
}

UserLogin.h
#ifndef USERLOGIN_H
#define USERLOGIN_H

#include <string>
#include <map>

using namespace std;

class UserLogin {
public:
    UserLogin(int maxUsers); // 构造函数，可以传入最大用户数量

```

```
int readMap(int adjacencyMatrix[][100], int maxNodes); //从文件中读取地图数据到邻接矩阵，假设最大节点数为 10
```

```
bool createUser(const string& username, const string& password); // 创建新用户（将用户名和密码存入用户数据结构）
```

```
bool verifyUser(const string& username, const string& password); // 验证用户名和密码是否匹配
```

```
private:
```

```
map<string, string> users; // 使用 map 来存储用户名和密码对
```

```
int maxUsers; // 最大用户数
```

```
};
```

```
#endif
```

UserLogin.cpp

```
#include "UserLogin.h"
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <iostream>
```

```
using namespace std;
```

```
UserLogin::UserLogin(int maxUsers) : maxUsers(maxUsers) {
```

```
    // 初始化用户数为 0
```

```
}
```

```
bool UserLogin::createUser(const string& username, const string& password) {
```

```
    if (users.size() < maxUsers) { // 判断当前用户数量是否小于最大用户数
```

```
        if (users.find(username) == users.end()) { // 确保用户名唯一
```

```
            users[username] = password; // 添加用户
```

```
            // 打开（或创建）用户文件
```

```
            ofstream file("D:/vscode_wenjian/SFML/" + username + ".txt");
```

```

        if (!file.is_open()) {
            cerr << "Failed to create user file.\n";
            return false;
        }

        // 写入密码到文件
        file << password << "\n"; // 第 1 行: 密码

        // 跳过第二行
        file << "\n"; // 第二行, 内容为空, 直接跳过

        file.close(); // 关闭文件
        cout << "User created successfully.\n";
        return true;
    } else {
        cerr << "User already exists.\n";
    }
}
return false; // 用户已存在或已达到最大用户数
}

bool UserLogin::verifyUser(const string& username, const string& password) {
    auto it = users.find(username);
    if (it != users.end() && it->second == password) {
        return true;
    }
    return false; // 用户名或密码不匹配
}

int UserLogin::readMap(int adjacencyMatrix[][100], int maxNodes) {
    ifstream file("user.txt");
    if (!file.is_open()) {
        cerr << "Failed to open map file!" << endl;
        return -1;
    }

    string line;
    int row = 0;

```

```
while (getline(file, line) && row < maxNodes) {
    stringstream ss(line);
    int value;
    int col = 0;

    while (ss >> value && col < maxNodes) {
        adjacencyMatrix[row][col] = value;
        col++;
    }
    row++;
}

file.close();
return 1;
}

main.cpp
#include <SFML/Graphics.hpp>
#include <SFML/Window.hpp>
#include <iostream>
#include <sstream>
#include <fstream>
#include "Graph.h"
#include "UserLogin.h"

using namespace std;

void drawMatrix(const Graph& graph, sf::RenderWindow& window, int n) {
    // 每个点的半径
    const float radius = 5.0f;

    // 点之间的间距
    const float spacing = 20.0f;

    // 创建点的形状
    sf::CircleShape point(radius);

    // 遍历邻接矩阵，绘制每个点
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
            // 设置点的颜色
            if (graph.edges[i][j] == 1) {
                point.setFillColor(sf::Color::Green); // 绿色表示 1
            } else {
                point.setFillColor(sf::Color::Red);    // 红色表示 0
            }

            // 计算点的位置
            float xPos = 100.f + j * spacing; // X 轴位置，按列排列
            float yPos = 100.f + i * spacing; // Y 轴位置，按行排列

            point.setPosition(xPos, yPos); // 设置点的位置

            // 绘制点
            window.draw(point);
        }
    }
}

int loadMapFromFile(const string& filename, Graph& graph) {
    ifstream file(filename); // 打开文件
    if (!file.is_open()) {
        cerr << "Failed to open file!" << endl;
        return -1; // 返回失败标识
    }

    string line;
    int row = 0;

    // 跳过前两行
    getline(file, line); // 第一行
    getline(file, line); // 第二行，读取成一维数组

    // 将第二行读取到一维数组
    int secondLineArray[graph.n];
    stringstream ss(line);
    int col = 0;
```

```
while (ss >> secondLineArray[col] && col < graph.n) {
    col++;
}

// 打印第二行数组（用于调试）
for (int i = 0; i < col; ++i) {
    cout << secondLineArray[i] << " ";
}
cout << endl;

// 从第三行开始读取邻接矩阵数据
while (getline(file, line) && row < graph.n) {
    stringstream ss(line);
    int col = 0;

    // 逐个读取行中的数字
    while (ss >> graph.edges[row][col] && col < graph.n) {
        col++;
    }
    row++;
}

file.close(); // 关闭文件
return 1; // 成功读取
}

void createErrorWindow(const string& errorMessage) {
    sf::RenderWindow errorWindow(sf::VideoMode(200, 100), "Error",
sf::Style::Close);

    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return;
    }

    sf::Text errorText(errorMessage, font, 18);
    errorText.setFillColor(sf::Color::Red);
    errorText.setPosition(20, 30);
}
```

```
while (errorWindow.isOpen()) {
    sf::Event event;
    while (errorWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            errorWindow.close();
        }
    }

    errorWindow.clear(sf::Color::White);
    errorWindow.draw(errorText);
    errorWindow.display();
}

}

void createLoginWindow(UserLogin& userLogin, sf::RenderWindow&
window, Graph& graph) {
    sf::RenderWindow loginWindow(sf::VideoMode(400, 300), "Login",
sf::Style::Close);

    // 创建字体
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return ;
    }

    // 创建文本框
    sf::RectangleShape usernameBox(sf::Vector2f(250, 30));
    usernameBox.setFill_color(sf::Color::White);
    usernameBox.setPosition(75, 50);

    sf::RectangleShape passwordBox(sf::Vector2f(250, 30));
    passwordBox.setFill_color(sf::Color::White);
    passwordBox.setPosition(75, 100);

    // 创建按钮
    sf::RectangleShape loginButton(sf::Vector2f(150, 40));
    loginButton.setFill_color(sf::Color::Green);
    loginButton.setPosition(50, 200);
}
```

```
sf::RectangleShape createAccountButton(sf::Vector2f(150, 40));
createAccountButton.setFillColor(sf::Color::Cyan);
createAccountButton.setPosition(205, 200);

// 创建按钮文字
sf::Text loginText("Login", font, 18);
loginText.setFillColor(sf::Color::Black);
loginText.setPosition(100, 210);

sf::Text createAccountText("Create Account", font, 18);
createAccountText.setFillColor(sf::Color::Black);
createAccountText.setPosition(215, 210);

// 创建用户名和密码的标签
sf::Text usernameLabel("Username:", font, 18);
usernameLabel.setFillColor(sf::Color::Black);
usernameLabel.setPosition(10, 50);

sf::Text passwordLabel("Password:", font, 18);
passwordLabel.setFillColor(sf::Color::Black);
passwordLabel.setPosition(10, 100);

// 用户输入
string usernameInput = "";
string passwordInput = "";
bool isUsernameFocused = true; // 当前输入框是否是用户名框

// 显示文本
sf::Text usernameText("", font, 18);
usernameText.setFillColor(sf::Color::Black);
usernameText.setPosition(100, 50);

sf::Text passwordText("", font, 18);
passwordText.setFillColor(sf::Color::Black);
passwordText.setPosition(100, 100);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
```



```

cursor.setFillColor(sf::Color::Black);
cursor.setPosition(100, 50);
bool cursorVisible = true; // 光标是否可见
sf::Clock cursorClock;      // 光标计时器

while (loginWindow.isOpen()) {
    sf::Event event;
    while (loginWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            loginWindow.close();
        }

        // 切换焦点
        if (event.type == sf::Event::MouseButtonPressed) {
            sf::Vector2i mousePos = sf::Mouse::getPosition(loginWindow);

            if (usernameBox.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                isUsernameFocused = true;
            } else if (passwordBox.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                isUsernameFocused = false;
            }

            // 检查点击的是“登录”按钮
            if (loginButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                if (userLogin.verifyUser(usernameInput, passwordInput)) {
                    cout << "Login successful" << endl;

                    int result =
loadMapFromFile("D:/vscode_wenjian/SFML/" + usernameInput + ".txt",
graph); // 读取地图数据

                    if (result == 1) {
                        cout << "Map read successfully!" << endl;
                        loginWindow.close(); // 关闭登录窗口
                        return; // 成功读取地图后退出
                    } else {
                        createErrorWindow("Error reading map");
                    }
                }
            }
        }
    }
}

```

```

        } else {
            // 如果密码错误，弹出错误提示窗口
            createErrorWindow("Wrong Password");
        }
    }

    // 检查点击的是“新建账号”按钮
                                                                    if
(createAccountButton.getGlobalBounds().contains(mousePos.x, mousePos.y)) {
        cout << "Create Account button clicked" << endl;
        if (userLogin.createUser(usernameInput, passwordInput))
        { // 调用 userLogin 的方法添加新用户
            cout << "Account created successfully!" << endl;
            createErrorWindow("Account Created");
        } else {
            createErrorWindow("Failed to Create Account");
        }
    }
}

// 处理用户输入
if (event.type == sf::Event::TextEntered) {
    if (event.text.unicode < 128) {
        char enteredChar = static_cast<char>(event.text.unicode);

        if (enteredChar == '\b') { // 退格键
            if (isUsernameFocused && !usernameInput.empty()) {
                usernameInput.pop_back();
            } else if (!isUsernameFocused
&& !passwordInput.empty()) {
                passwordInput.pop_back();
            }
        } else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符

            if (isUsernameFocused) {
                usernameInput += enteredChar;
            } else {
                passwordInput += enteredChar;
            }
        }
    }
}

```

```

        }
    }
}

// 更新文本和光标位置
usernameText.setString(usernameInput);
passwordText.setString(string(passwordInput.size(), '*')); // 显示为星号

if (isUsernameFocused) {
    cursor.setPosition(100 + usernameText.getGlobalBounds().width, 50);
} else {
    cursor.setPosition(100 + passwordText.getGlobalBounds().width,
100);
}

// 光标闪烁
if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
    cursorVisible = !cursorVisible;
    cursorClock.restart();
}

// 清空和绘制窗口
loginWindow.clear(sf::Color::White);

// 绘制所有元素
loginWindow.draw(usernameBox);
loginWindow.draw(passwordBox);
loginWindow.draw(loginButton);
loginWindow.draw(createAccountButton);
loginWindow.draw(usernameLabel);
loginWindow.draw(passwordLabel);
loginWindow.draw(loginText);
loginWindow.draw(createAccountText);
loginWindow.draw(usernameText);
loginWindow.draw(passwordText);

// 绘制光标
if (cursorVisible) {

```

```
        loginWindow.draw(cursor);
    }

    loginWindow.display();
}
}

void createAddSideWindow(Graph& graph, sf::RenderWindow& window) {
    sf::RenderWindow addSideWindow(sf::VideoMode(400, 300), "Add Side",
sf::Style::Close);

    // 创建字体
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return;
    }

    // 创建文本框
    sf::RectangleShape uBox(sf::Vector2f(250, 30));
    uBox.setFillColor(sf::Color::White);
    uBox.setPosition(75, 50);

    sf::RectangleShape vBox(sf::Vector2f(250, 30));
    vBox.setFillColor(sf::Color::White);
    vBox.setPosition(75, 100);

    // 创建按钮
    sf::RectangleShape yesButton(sf::Vector2f(150, 40));
    yesButton.setFillColor(sf::Color::Green);
    yesButton.setPosition(125, 200);

    // 创建按钮文字
    sf::Text yesText("Yes", font, 18);
    yesText.setFillColor(sf::Color::Black);
    yesText.setPosition(175, 210);

    // 创建标签
    sf::Text uLabel("Node u:", font, 18);
```

```

uLabel.setFillColor(sf::Color::Black);
uLabel.setPosition(10, 50);

sf::Text vLabel("Node v:", font, 18);
vLabel.setFillColor(sf::Color::Black);
vLabel.setPosition(10, 100);

// 用户输入
string uInput = "";
string vInput = "";
bool isUFocused = true; // 当前输入框是否是 u 框

// 显示文本
sf::Text uText("", font, 18);
uText.setFillColor(sf::Color::Black);
uText.setPosition(100, 50);

sf::Text vText("", font, 18);
vText.setFillColor(sf::Color::Black);
vText.setPosition(100, 100);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
cursor.setFillColor(sf::Color::Black);
cursor.setPosition(100, 50);
bool cursorVisible = true;
sf::Clock cursorClock;

while (addSideWindow.isOpen()) {
    sf::Event event;
    while (addSideWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            addSideWindow.close();
        }

        // 切换焦点
        if (event.type == sf::Event::MouseButtonPressed) {
            sf::Vector2i mousePos =
sf::Mouse::getPosition(addSideWindow);

```

```

        if (uBox.getGlobalBounds().contains(mousePos.x, mousePos.y))
        {
            isUFocused = true;
        } else if (vBox.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
            isUFocused = false;
        }

        // 检查点击的是“Yes”按钮
        if (yesButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
            if (!uInput.empty() && !vInput.empty()) {
                // 将 u 和 v 转换为整数并添加边
                int u = stoi(uInput);
                int v = stoi(vInput);
                graph.addEdge(u, v); // 假设 addEdge 是 Graph 类
的方法

                addSideWindow.close(); // 关闭窗口
            } else {
                // 提示用户输入有效的节点
                createErrorWindow("Invalid input!");
            }
        }
    }

    // 处理用户输入
    if (event.type == sf::Event::TextEntered) {
        if (event.text.unicode < 128) {
            char enteredChar = static_cast<char>(event.text.unicode);

            if (enteredChar == '\b') { // 退格键
                if (isUFocused && !uInput.empty()) {
                    uInput.pop_back();
                } else if (!isUFocused && !vInput.empty()) {
                    vInput.pop_back();
                }
            } else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符

```

```
        if (isUFocused) {
            uInput += enteredChar;
        } else {
            vInput += enteredChar;
        }
    }
}

// 更新文本和光标位置
uText.setString(uInput);
vText.setString(vInput);

if (isUFocused) {
    cursor.setPosition(100 + uText.getGlobalBounds().width, 50);
} else {
    cursor.setPosition(100 + vText.getGlobalBounds().width, 100);
}

// 光标闪烁
if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
    cursorVisible = !cursorVisible;
    cursorClock.restart();
}

// 清空和绘制窗口
addSideWindow.clear(sf::Color::White);

// 绘制所有元素
addSideWindow.draw(uBox);
addSideWindow.draw(vBox);
addSideWindow.draw(yesButton);
addSideWindow.draw(uLabel);
addSideWindow.draw(vLabel);
addSideWindow.draw(yesText);
addSideWindow.draw(uText);
addSideWindow.draw(vText);
```

```
// 绘制光标
if (cursorVisible) {
    addSideWindow.draw(cursor);
}

addSideWindow.display();
}
}

void createDeleteSideWindow(Graph& graph, sf::RenderWindow& window) {
    sf::RenderWindow deleteSideWindow(sf::VideoMode(400, 300), "Delete Side",
    sf::Style::Close);

    // 创建字体
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return;
    }

    // 创建文本框
    sf::RectangleShape uBox(sf::Vector2f(250, 30));
    uBox.setFillColor(sf::Color::White);
    uBox.setPosition(75, 50);

    sf::RectangleShape vBox(sf::Vector2f(250, 30));
    vBox.setFillColor(sf::Color::White);
    vBox.setPosition(75, 100);

    // 创建按钮
    sf::RectangleShape yesButton(sf::Vector2f(150, 40));
    yesButton.setFillColor(sf::Color::Red);
    yesButton.setPosition(125, 200);

    // 创建按钮文字
    sf::Text yesText("Yes", font, 18);
    yesText.setFillColor(sf::Color::Black);
    yesText.setPosition(175, 210);
```



```
// 创建标签
sf::Text uLabel("Node u:", font, 18);
uLabel.setFillColor(sf::Color::Black);
uLabel.setPosition(10, 50);

sf::Text vLabel("Node v:", font, 18);
vLabel.setFillColor(sf::Color::Black);
vLabel.setPosition(10, 100);

// 用户输入
string uInput = "";
string vInput = "";
bool isUFocused = true; // 当前输入框是否是 u 框

// 显示文本
sf::Text uText("", font, 18);
uText.setFillColor(sf::Color::Black);
uText.setPosition(100, 50);

sf::Text vText("", font, 18);
vText.setFillColor(sf::Color::Black);
vText.setPosition(100, 100);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
cursor.setFillColor(sf::Color::Black);
cursor.setPosition(100, 50);
bool cursorVisible = true;
sf::Clock cursorClock;

while (deleteSideWindow.isOpen()) {
    sf::Event event;
    while (deleteSideWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            deleteSideWindow.close();
        }

        // 切换焦点
        if (event.type == sf::Event::MouseButtonPressed) {
```

```

sf::Vector2i mousePos =
sf::Mouse::getPosition(deleteSideWindow);

    if (uBox.getGlobalBounds().contains(mousePos.x, mousePos.y))
    {
        isUFocused = true;
    } else if (vBox.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
        isUFocused = false;
    }

    // 检查点击的是“Yes”按钮
    if (yesButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
        if (!uInput.empty() && !vInput.empty()) {
            // 将 u 和 v 转换为整数并删除边
            int u = stoi(uInput);
            int v = stoi(vInput);
            graph.removeEdge(u, v); // 假设 deleteEdge 是
Graph 类的方法

            deleteSideWindow.close(); // 关闭窗口
        } else {
            // 提示用户输入有效的节点
            createErrorWindow("Invalid input!");
        }
    }
}

// 处理用户输入
if (event.type == sf::Event::TextEntered) {
    if (event.text.unicode < 128) {
        char enteredChar = static_cast<char>(event.text.unicode);

        if (enteredChar == '\b') { // 退格键
            if (isUFocused && !uInput.empty()) {
                uInput.pop_back();
            } else if (!isUFocused && !vInput.empty()) {
                vInput.pop_back();
            }
        }
    }
}

```

```

    } else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符
        if (isUFocused) {
            uInput += enteredChar;
        } else {
            vInput += enteredChar;
        }
    }
}

// 更新文本和光标位置
uText.setString(uInput);
vText.setString(vInput);

if (isUFocused) {
    cursor.setPosition(100 + uText.getGlobalBounds().width, 50);
} else {
    cursor.setPosition(100 + vText.getGlobalBounds().width, 100);
}

// 光标闪烁
if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
    cursorVisible = !cursorVisible;
    cursorClock.restart();
}

// 清空和绘制窗口
deleteSideWindow.clear(sf::Color::White);

// 绘制所有元素
deleteSideWindow.draw(uBox);
deleteSideWindow.draw(vBox);
deleteSideWindow.draw(yesButton);
deleteSideWindow.draw(uLabel);
deleteSideWindow.draw(vLabel);
deleteSideWindow.draw(yesText);
deleteSideWindow.draw(uText);

```

```
deleteSideWindow.draw(vText);

// 绘制光标
if (cursorVisible) {
    deleteSideWindow.draw(cursor);
}

deleteSideWindow.display();
}
}

void createSearchShortestPathWindow(Graph& graph, sf::RenderWindow&
window) {
    sf::RenderWindow searchPathWindow(sf::VideoMode(400, 300), "Search
Shortest Path", sf::Style::Close);

    // 创建字体
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return;
    }

    // 创建文本框
    sf::RectangleShape uBox(sf::Vector2f(250, 30));
    uBox.setFillColor(sf::Color::White);
    uBox.setPosition(75, 50);

    sf::RectangleShape vBox(sf::Vector2f(250, 30));
    vBox.setFillColor(sf::Color::White);
    vBox.setPosition(75, 100);

    // 创建按钮
    sf::RectangleShape yesButton(sf::Vector2f(150, 40));
    yesButton.setFillColor(sf::Color::Blue);
    yesButton.setPosition(125, 200);

    // 创建按钮文字
    sf::Text yesText("Yes", font, 18);
```

```

yesText.setFillColor(sf::Color::White);
yesText.setPosition(175, 210);

// 创建标签
sf::Text uLabel("Start Node u:", font, 18);
uLabel.setFillColor(sf::Color::Black);
uLabel.setPosition(10, 50);

sf::Text vLabel("End Node v:", font, 18);
vLabel.setFillColor(sf::Color::Black);
vLabel.setPosition(10, 100);

// 用户输入
string uInput = "";
string vInput = "";
bool isUFocused = true; // 当前输入框是否是 u 框

// 显示文本
sf::Text uText("", font, 18);
uText.setFillColor(sf::Color::Black);
uText.setPosition(120, 50);

sf::Text vText("", font, 18);
vText.setFillColor(sf::Color::Black);
vText.setPosition(120, 100);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
cursor.setFillColor(sf::Color::Black);
cursor.setPosition(120, 50);
bool cursorVisible = true;
sf::Clock cursorClock;

while (searchPathWindow.isOpen()) {
    sf::Event event;
    while (searchPathWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            searchPathWindow.close();
        }
    }
}

```

```

// 切换焦点
if (event.type == sf::Event::MouseButtonPressed) {
    sf::Vector2i mousePos =
sf::Mouse::getPosition(searchPathWindow);

    if (uBox.getGlobalBounds().contains(mousePos.x, mousePos.y))
    {
        isUFocused = true;
    } else if (vBox.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
        isUFocused = false;
    }

    // 检查点击的是“Yes”按钮
    if (yesButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
        if (!uInput.empty() && !vInput.empty()) {
            // 转换为整数并查找最短路径
            int u = stoi(uInput);
            int v = stoi(vInput);
            string result = graph.shortestPath(u, v); // 假设返回路
径或错误信息

            createErrorWindow(result);
            searchPathWindow.close(); // 关闭窗口
        } else {
            // 提示用户输入有效的节点
            createErrorWindow("Invalid input!");
        }
    }
}

// 处理用户输入
if (event.type == sf::Event::TextEntered) {
    if (event.text.unicode < 128) {
        char enteredChar = static_cast<char>(event.text.unicode);

        if (enteredChar == '\b') { // 退格键
            if (isUFocused && !uInput.empty()) {

```

```

        uInput.pop_back();
    } else if (!isUFocused && !vInput.empty()) {
        vInput.pop_back();
    }
} else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符

    if (isUFocused) {
        uInput += enteredChar;
    } else {
        vInput += enteredChar;
    }
}

}

}

// 更新文本和光标位置
uText.setString(uInput);
vText.setString(vInput);

if (isUFocused) {
    cursor.setPosition(120 + uText.getGlobalBounds().width, 50);
} else {
    cursor.setPosition(120 + vText.getGlobalBounds().width, 100);
}

// 光标闪烁
if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
    cursorVisible = !cursorVisible;
    cursorClock.restart();
}

// 清空和绘制窗口
searchPathWindow.clear(sf::Color::White);

// 绘制所有元素
searchPathWindow.draw(uBox);
searchPathWindow.draw(vBox);
searchPathWindow.draw(yesButton);

```

```
searchPathWindow.draw(uLabel);
searchPathWindow.draw(vLabel);
searchPathWindow.draw(yesText);
searchPathWindow.draw(uText);
searchPathWindow.draw(vText);

// 绘制光标
if (cursorVisible) {
    searchPathWindow.draw(cursor);
}

searchPathWindow.display();
}
}

void createSearchAllPathsWindow(Graph& graph, sf::RenderWindow& window) {
    sf::RenderWindow searchPathsWindow(sf::VideoMode(400, 300), "Search All
Paths", sf::Style::Close);

    // 创建字体
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return;
    }

    // 创建文本框
    sf::RectangleShape uBox(sf::Vector2f(250, 30));
    uBox.setFillColor(sf::Color::White);
    uBox.setPosition(75, 50);

    sf::RectangleShape vBox(sf::Vector2f(250, 30));
    vBox.setFillColor(sf::Color::White);
    vBox.setPosition(75, 100);

    // 创建按钮
    sf::RectangleShape yesButton(sf::Vector2f(150, 40));
    yesButton.setFillColor(sf::Color::Magenta);
    yesButton.setPosition(125, 200);
```



```
// 创建按钮文字
sf::Text yesText("Yes", font, 18);
yesText.setFillColor(sf::Color::White);
yesText.setPosition(175, 210);

// 创建标签
sf::Text uLabel("Start Node u:", font, 18);
uLabel.setFillColor(sf::Color::Black);
uLabel.setPosition(10, 50);

sf::Text vLabel("End Node v:", font, 18);
vLabel.setFillColor(sf::Color::Black);
vLabel.setPosition(10, 100);

// 用户输入
string uInput = "";
string vInput = "";
bool isUFocused = true; // 当前输入框是否是 u 框

// 显示文本
sf::Text uText("", font, 18);
uText.setFillColor(sf::Color::Black);
uText.setPosition(120, 50);

sf::Text vText("", font, 18);
vText.setFillColor(sf::Color::Black);
vText.setPosition(120, 100);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
cursor.setFillColor(sf::Color::Black);
cursor.setPosition(120, 50);
bool cursorVisible = true;
sf::Clock cursorClock;

while (searchPathsWindow.isOpen()) {
    sf::Event event;
    while (searchPathsWindow.pollEvent(event)) {
```

```

        if (event.type == sf::Event::Closed) {
            searchPathsWindow.close();
        }

        // 切换焦点
        if (event.type == sf::Event::MouseButtonPressed) {
            sf::Vector2i mousePos =
sf::Mouse::getPosition(searchPathsWindow);

            if (uBox.getGlobalBounds().contains(mousePos.x, mousePos.y))
{
                isUFocused = true;
            } else if (vBox.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                isUFocused = false;
            }

            // 检查点击的是“Yes”按钮
            if (yesButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                if (!uInput.empty() && !vInput.empty()) {
                    // 转换为整数并查找所有路径
                    int u = stoi(uInput);
                    int v = stoi(vInput);
                    string result = graph.searchAllPaths(u, v); // 假设返回
所有路径信息或错误信息
                    createErrorWindow(result);
                    searchPathsWindow.close(); // 关闭窗口
                } else {
                    // 提示用户输入有效的节点
                    createErrorWindow("Invalid input!");
                }
            }
        }

        // 处理用户输入
        if (event.type == sf::Event::TextEntered) {
            if (event.text.unicode < 128) {
                char enteredChar = static_cast<char>(event.text.unicode);

```

```

        if (enteredChar == '\b') { // 退格键
            if (isUFocused && !uInput.empty()) {
                uInput.pop_back();
            } else if (!isUFocused && !vInput.empty()) {
                vInput.pop_back();
            }
        } else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符
        if (isUFocused) {
            uInput += enteredChar;
        } else {
            vInput += enteredChar;
        }
    }
}

// 更新文本和光标位置
uText.setString(uInput);
vText.setString(vInput);

if (isUFocused) {
    cursor.setPosition(120+ uText.getGlobalBounds().width, 50);
} else {
    cursor.setPosition(120 + vText.getGlobalBounds().width, 100);
}

// 光标闪烁
if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
    cursorVisible = !cursorVisible;
    cursorClock.restart();
}

// 清空和绘制窗口
searchPathsWindow.clear(sf::Color::White);

// 绘制所有元素

```

```

        searchPathsWindow.draw(uBox);
        searchPathsWindow.draw(vBox);
        searchPathsWindow.draw(yesButton);
        searchPathsWindow.draw(uLabel);
        searchPathsWindow.draw(vLabel);
        searchPathsWindow.draw(yesText);
        searchPathsWindow.draw(uText);
        searchPathsWindow.draw(vText);

        // 绘制光标
        if (cursorVisible) {
            searchPathsWindow.draw(cursor);
        }

        searchPathsWindow.display();
    }
}

void createDeleteDotWindow(Graph& graph, sf::RenderWindow& window) {
    sf::RenderWindow deleteDotWindow(sf::VideoMode(400, 200), "Delete Dot",
    sf::Style::Close);

    // 创建字体
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return;
    }

    // 创建文本框
    sf::RectangleShape uBox(sf::Vector2f(250, 30));
    uBox.setFillColor(sf::Color::White);
    uBox.setPosition(75, 50);

    // 创建按钮
    sf::RectangleShape yesButton(sf::Vector2f(150, 40));
    yesButton.setFillColor(sf::Color::Red);
    yesButton.setPosition(125, 120);

```

```

// 创建按钮文字
sf::Text yesText("Yes", font, 18);
yesText.setFillColor(sf::Color::White);
yesText.setPosition(175, 130);

// 创建标签
sf::Text uLabel("Node to delete u:", font, 18);
uLabel.setFillColor(sf::Color::Black);
uLabel.setPosition(10, 50);

// 用户输入
string uInput = "";
bool isUFocused = true; // 当前输入框是否是 u 框

// 显示文本
sf::Text uText("", font, 18);
uText.setFillColor(sf::Color::Black);
uText.setPosition(145, 50);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
cursor.setFillColor(sf::Color::Black);
cursor.setPosition(145, 50);
bool cursorVisible = true;
sf::Clock cursorClock;

while (deleteDotWindow.isOpen()) {
    sf::Event event;
    while (deleteDotWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            deleteDotWindow.close();
        }

        // 切换焦点
        if (event.type == sf::Event::MouseButtonPressed) {
            sf::Vector2i mousePos =
sf::Mouse::getPosition(deleteDotWindow);

            if (uBox.getGlobalBounds().contains(mousePos.x, mousePos.y))

```

```

{
    isUFocused = true;
}

// 检查点击的是“Yes”按钮
if (yesButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
    if (!uInput.empty()) {
        // 转换为整数并删除节点
        int u = stoi(uInput);
        bool result = graph.deleteNode(u); // 假设返回删除成
功或失败

        if (result) {
            createErrorWindow("Node deleted
successfully!");
        } else {
            createErrorWindow("Failed to delete node.");
        }
        deleteDotWindow.close(); // 关闭窗口
    } else {
        // 提示用户输入有效的节点
        createErrorWindow("Invalid input!");
    }
}

// 处理用户输入
if (event.type == sf::Event::TextEntered) {
    if (event.text.unicode < 128) {
        char enteredChar = static_cast<char>(event.text.unicode);

        if (enteredChar == '\b') { // 退格键
            if (isUFocused && !uInput.empty()) {
                uInput.pop_back();
            }
        } else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符

            if (isUFocused) {
                uInput += enteredChar;
            }
        }
    }
}

```

```

        }
    }
}

// 更新文本和光标位置
uText.setString(uInput);

cursor.setPosition(145 + uText.getGlobalBounds().width, 50);

// 光标闪烁
if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
    cursorVisible = !cursorVisible;
    cursorClock.restart();
}

// 清空和绘制窗口
deleteDotWindow.clear(sf::Color::White);

// 绘制所有元素
deleteDotWindow.draw(uBox);
deleteDotWindow.draw(yesButton);
deleteDotWindow.draw(uLabel);
deleteDotWindow.draw(yesText);
deleteDotWindow.draw(uText);

// 绘制光标
if (cursorVisible) {
    deleteDotWindow.draw(cursor);
}

deleteDotWindow.display();
}
}

void createAddDotWindow(Graph& graph, sf::RenderWindow& window) {
    sf::RenderWindow addDotWindow(sf::VideoMode(400, 200), "Add Dot",
    sf::Style::Close);

```

```
// 创建字体
sf::Font font;
if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
    cerr << "Failed to load font!" << endl;
    return;
}

// 创建文本框
sf::RectangleShape uBox(sf::Vector2f(250, 30));
uBox.setFillColor(sf::Color::White);
uBox.setPosition(75, 50);

// 创建按钮
sf::RectangleShape yesButton(sf::Vector2f(150, 40));
yesButton.setFillColor(sf::Color::Blue);
yesButton.setPosition(125, 120);

// 创建按钮文字
sf::Text yesText("Yes", font, 18);
yesText.setFillColor(sf::Color::White);
yesText.setPosition(175, 130);

// 创建标签
sf::Text uLabel("Node Value u:", font, 18);
uLabel.setFillColor(sf::Color::Black);
uLabel.setPosition(10, 50);

// 用户输入
string uInput = "";
bool isUFocused = true; // 当前焦点是否在 u 框

// 显示文本
sf::Text uText("", font, 18);
uText.setFillColor(sf::Color::Black);
uText.setPosition(120, 50);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
```



```

cursor.setFillColor(sf::Color::Black);
cursor.setPosition(120, 50);
bool cursorVisible = true;
sf::Clock cursorClock;

while (addDotWindow.isOpen()) {
    sf::Event event;
    while (addDotWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            addDotWindow.close();
        }

        // 切换焦点
        if (event.type == sf::Event::MouseButtonPressed) {
            sf::Vector2i mousePos =
sf::Mouse::getPosition(addDotWindow);

            if (uBox.getGlobalBounds().contains(mousePos.x, mousePos.y))
{
                isUFocused = true;
            }

            // 检查点击的是“Yes”按钮
            if (yesButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                if (!uInput.empty()) {
                    int u = stoi(uInput);
                    if (graph.addDot(u)) { // 假设返回布尔值表示添加
成功
                        createErrorWindow("Node added successfully!");
                    } else {
                        createErrorWindow("Node already exists or
invalid input!");
                    }
                }
                addDotWindow.close();
            } else {
                createErrorWindow("Invalid input!");
            }
        }
    }
}

```

```

    }

    // 处理用户输入
    if (event.type == sf::Event::TextEntered) {
        if (event.text.unicode < 128) {
            char enteredChar = static_cast<char>(event.text.unicode);

            if (enteredChar == '\b') { // 退格键
                if (!uInput.empty()) {
                    uInput.pop_back();
                }
            } else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符

                uInput += enteredChar;
            }
        }
    }

    // 更新文本和光标位置
    uText.setString(uInput);
    cursor.setPosition(120 + uText.getGlobalBounds().width, 50);

    // 光标闪烁
    if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
        cursorVisible = !cursorVisible;
        cursorClock.restart();
    }

    // 清空和绘制窗口
    addDotWindow.clear(sf::Color::White);

    // 绘制所有元素
    addDotWindow.draw(uBox);
    addDotWindow.draw(yesButton);
    addDotWindow.draw(uLabel);
    addDotWindow.draw(yesText);
    addDotWindow.draw(uText);

```

```
// 绘制光标
if (cursorVisible) {
    addDotWindow.draw(cursor);
}

addDotWindow.display();
}
}

void createSearchAdjacentDotWindow(Graph& graph, sf::RenderWindow&
window) {
    sf::RenderWindow searchAdjacentWindow(sf::VideoMode(400, 200), "Search
Adjacent Dot", sf::Style::Close);

    // 创建字体
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return;
    }

    // 创建文本框
    sf::RectangleShape uBox(sf::Vector2f(250, 30));
    uBox.setFillColor(sf::Color::White);
    uBox.setPosition(75, 50);

    // 创建按钮
    sf::RectangleShape searchButton(sf::Vector2f(150, 40));
    searchButton.setFillColor(sf::Color::Green);
    searchButton.setPosition(125, 120);

    // 创建按钮文字
    sf::Text searchText("Search", font, 18);
    searchText.setFillColor(sf::Color::White);
    searchText.setPosition(175, 130);

    // 创建标签
    sf::Text uLabel("Node Value u:", font, 18);
    uLabel.setFillColor(sf::Color::Black);
```

```

uLabel.setPosition(10, 50);

// 用户输入
string uInput = "";
bool isUFocused = true; // 当前焦点是否在 u 框

// 显示文本
sf::Text uText("", font, 18);
uText.setFillColor(sf::Color::Black);
uText.setPosition(120, 50);

// 光标
sf::RectangleShape cursor(sf::Vector2f(2, 25));
cursor.setFillColor(sf::Color::Black);
cursor.setPosition(120, 50);
bool cursorVisible = true;
sf::Clock cursorClock;

while (searchAdjacentWindow.isOpen()) {
    sf::Event event;
    while (searchAdjacentWindow.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            searchAdjacentWindow.close();
        }

        // 切换焦点
        if (event.type == sf::Event::MouseButtonPressed) {
            sf::Vector2i mousePos =
sf::Mouse::getPosition(searchAdjacentWindow);

            if (uBox.getGlobalBounds().contains(mousePos.x, mousePos.y))
{
                isUFocused = true;
            }

            // 检查点击的是“Search”按钮
            if (searchButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                if (!uInput.empty()) {

```

```

        int u = stoi(uInput);
        string result = graph.adjacentVertices(u); // 调用 Graph
类的 adjacentVertices 方法
        createErrorWindow(result);
        searchAdjacentWindow.close();
    } else {
        createErrorWindow("Invalid input!");
    }
}

// 处理用户输入
if (event.type == sf::Event::TextEntered) {
    if (event.text.unicode < 128) {
        char enteredChar = static_cast<char>(event.text.unicode);

        if (enteredChar == '\b') { // 退格键
            if (!uInput.empty()) {
                uInput.pop_back();
            }
        } else if (enteredChar >= 32 && enteredChar <= 126) { //
可打印字符
            uInput += enteredChar;
        }
    }
}

// 更新文本和光标位置
uText.setString(uInput);
cursor.setPosition(120 + uText.getGlobalBounds().width, 50);

// 光标闪烁
if (cursorClock.getElapsedTime().asSeconds() > 0.5f) {
    cursorVisible = !cursorVisible;
    cursorClock.restart();
}

// 清空和绘制窗口

```

```
searchAdjacentWindow.clear(sf::Color::White);

// 绘制所有元素
searchAdjacentWindow.draw(uBox);
searchAdjacentWindow.draw(searchButton);
searchAdjacentWindow.draw(uLabel);
searchAdjacentWindow.draw(searchText);
searchAdjacentWindow.draw(uText);

// 绘制光标
if (cursorVisible) {
    searchAdjacentWindow.draw(cursor);
}

searchAdjacentWindow.display();
}
}

int main() {
    UserLogin userLogin(5);
    Graph graph; // 创建一个图对象
    string username = "user"; // 假设这是从 UserLogin 获取的用户名
    string filename = "D:/vscode_wenjian/SFML/" + username + ".txt";
    graph.loadGraphFromFile(filename);

    sf::RenderWindow window(sf::VideoMode(800, 600), "SFML Window");

    window.setFramerateLimit(60);
    sf::Font font;
    if (!font.loadFromFile("D:/vscode_wenjian/SFML/arial.ttf")) {
        cerr << "Failed to load font!" << endl;
        return -1;
    }

    // 创建右上角的圆形按钮
    sf::CircleShape loginButton(25); // 半径为 25，即直径为 50
    loginButton.setFillColor(sf::Color::Blue);
    loginButton.setPosition(725, 25); // 设置位置（右上角）
```

```

        sf::RectangleShape deleteSideButton(sf::Vector2f(100, 50)); // 按钮尺寸
        100*50
        deleteSideButton.setFillColor(sf::Color::Green);
        deleteSideButton.setPosition(25, 25); // 设置位置

        sf::RectangleShape addSideButton(sf::Vector2f(100, 50));
        addSideButton.setFillColor(sf::Color::Cyan);
        addSideButton.setPosition(127, 25);

        sf::RectangleShape deleteDotButton(sf::Vector2f(100, 50));
        deleteDotButton.setFillColor(sf::Color::Green);
        deleteDotButton.setPosition(229, 25);

        sf::RectangleShape addDotButton(sf::Vector2f(100, 50));
        addDotButton.setFillColor(sf::Color::Cyan);
        addDotButton.setPosition(331, 25);

        sf::RectangleShape searchshortestpathButton(sf::Vector2f(140, 50));
        searchshortestpathButton.setFillColor(sf::Color::Green);
        searchshortestpathButton.setPosition(440, 25);

        sf::RectangleShape searchallpathButton(sf::Vector2f(120, 50));
        searchallpathButton.setFillColor(sf::Color::Cyan);
        searchallpathButton.setPosition(585, 25);

        sf::RectangleShape searchadjacentdotButton(sf::Vector2f(170, 50));
        searchadjacentdotButton.setFillColor(sf::Color::Cyan);
        searchadjacentdotButton.setPosition(585, 85);

        // 创建按钮文字
        sf::Text loginText("LOG", font, 18);
        loginText.setFillColor(sf::Color::White);
        loginText.setPosition(731, 40);

        sf::Text deleteSideText("Delete Side", font, 18);
        deleteSideText.setFillColor(sf::Color::Black);
        deleteSideText.setPosition(28, 40);

        sf::Text addSideText("Add Side", font, 18);
        addSideText.setFillColor(sf::Color::Black);
    
```

```

addSideText.setPosition(140, 40);

sf::Text deleteDotText("Delete Dot", font, 18);
deleteDotText.setFillColor(sf::Color::Black);
deleteDotText.setPosition(238, 40);

sf::Text addDotText("Add Dot", font, 18);
addDotText.setFillColor(sf::Color::Black);
addDotText.setPosition(343, 40);

sf::Text searchshortestpathText("SearchShortestP", font, 18);
searchshortestpathText.setFillColor(sf::Color::Black);
searchshortestpathText.setPosition(445, 40);

sf::Text searchallpathText("Search All P", font, 18);
searchallpathText.setFillColor(sf::Color::Black);
searchallpathText.setPosition(595, 40);

sf::Text searchadjacentdotText("Search Adjacent D", font, 18);
searchadjacentdotText.setFillColor(sf::Color::Black);
searchadjacentdotText.setPosition(595, 100);

while (window.isOpen()) {
    sf::Event event;
    while (window.pollEvent(event)) {
        if (event.type == sf::Event::Closed) {
            window.close();
        }
    }

    // 处理点击登录按钮
    if (event.type == sf::Event::MouseButtonPressed) {
        sf::Vector2i mousePos = sf::Mouse::getPosition(window);
        if (loginButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
            createLoginWindow(userLogin, window, graph); // 点击
登录按钮时弹出登录窗口
        }
    }
}

```



```

        if (event.type == sf::Event::MouseButtonPressed) {
            sf::Vector2i mousePos = sf::Mouse::getPosition(window);

            // 检查点击的是“Add Side”按钮
            if (addSideButton.getGlobalBounds().contains(mousePos.x, mousePos.y)) {
                createAddSideWindow(graph, window);
            }

            // 检查点击的是“Delete Side”按钮
            if (deleteSideButton.getGlobalBounds().contains(mousePos.x, mousePos.y)) {
                createDeleteSideWindow(graph, window);
            }

            // 检查点击的是“Delete Dot”按钮
            if (deleteDotButton.getGlobalBounds().contains(mousePos.x, mousePos.y)) {
                createDeleteDotWindow(graph, window);
            }

            // 检查点击的是“Add Dot”按钮
            if (addDotButton.getGlobalBounds().contains(mousePos.x, mousePos.y)) {
                createAddDotWindow(graph, window);
            }

            // 检查点击的是“Search Shortest Path”按钮
            if (searchshortestpathButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                createSearchShortestPathWindow(graph, window);
            }

            // 检查点击的是“Search All Paths”按钮
            if (searchallpathButton.getGlobalBounds().contains(mousePos.x, mousePos.y))
            {
                createSearchAllPathsWindow(graph, window);
            }

            // 检查点击的是“searchadjacentdotButton”按钮
            if (searchadjacentdotButton.getGlobalBounds().contains(mousePos.x,
mousePos.y)) {
                createSearchAdjacentDotWindow(graph, window);
            }
        }
    }
}

```

```
}  
}  
  
    window.clear(sf::Color::White);  
  
    // 在主窗口中绘制矩阵  
    drawMatrix(graph,window,graph.n);  
  
    // 绘制按钮  
    window.draw(loginButton);           // 绘制右上角的按钮  
    window.draw(deleteSideButton);  
    window.draw(addSideButton);  
    window.draw(deleteDotButton);  
    window.draw(addDotButton);  
    window.draw(searchshortestpathButton);  
    window.draw(searchallpathButton);  
    window.draw(searchadjacentdotButton);  
    window.draw(deleteSideText);  
    window.draw(addSideText);  
    window.draw(deleteDotText);  
    window.draw(addDotText);  
    window.draw(loginText);  
    window.draw(searchshortestpathText);  
    window.draw(searchallpathText);  
    window.draw(searchadjacentdotText);  
  
    window.display();  
}  
  
    return 0;  
}
```